

# CSCI 598H

## Applied Computer Vision for Engineering Systems

### Lecture 5-1: Attention and Transformer

**Dr. Minghan Li**

**Assistant Professor**

**Computer Science Department**

**Colorado School of Mines**

**Aug/27/2026**



# Administration

- Assignment 1: extended deadline **Sept/27 11.59pm**
- **#2 Code Section: Sept/25 (this Friday) 12pm-1pm, Marquez 204**

# CSCI 598H Course Overview



**Weeks 2-5**

Weeks 5-11

Weeks 12-16

## Module 1

### Deep Learning Basics

- Linear Classifier and Optimization
- CNNs and BackPropagation
- RNNs, **Attention**, **Transformers**

## Module 2

### Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

## Module 3

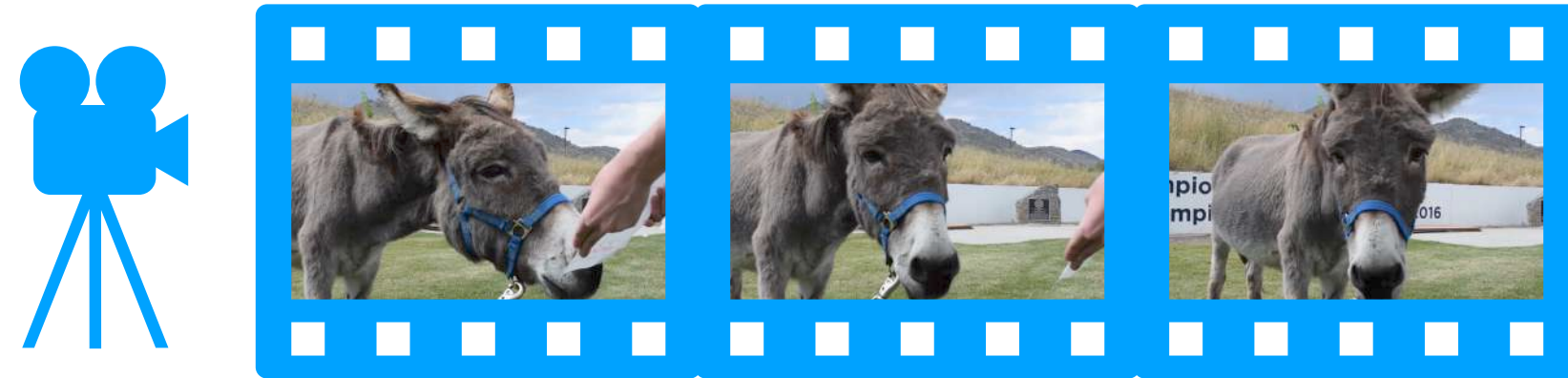
### Engineering Systems

- Robotics
- Earth science
- Medical imaging
- Energy

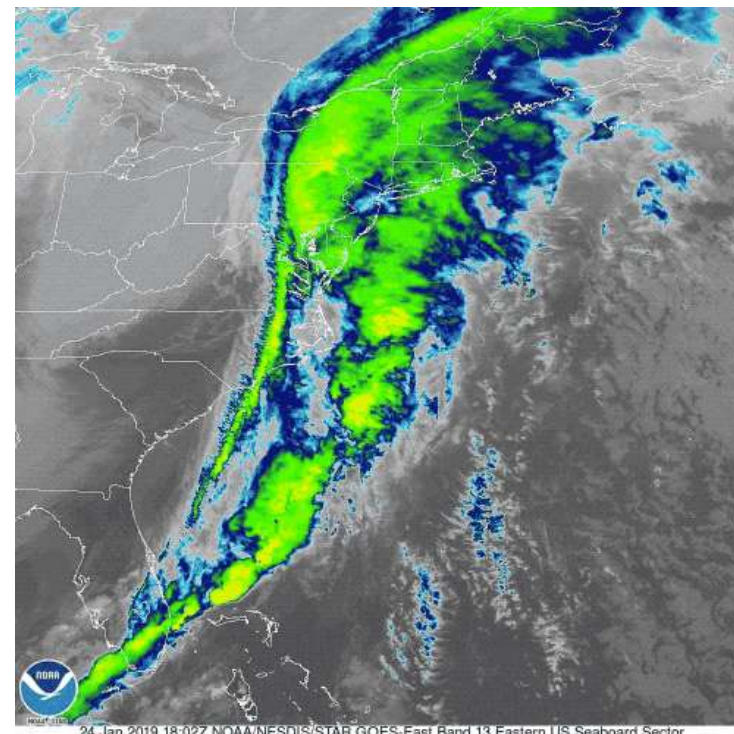
# Recap

## **W4-1: Recurrent Neural Networks**

# Recap: Sequence data and modeling



Video: *sequences of words, tokens*



Weather and climate data:  
*temperature, precipitation, wind*

<https://www.sciencefriday.com/segments/100-years-of-your-daily-weather-forecast/>

I love this class!

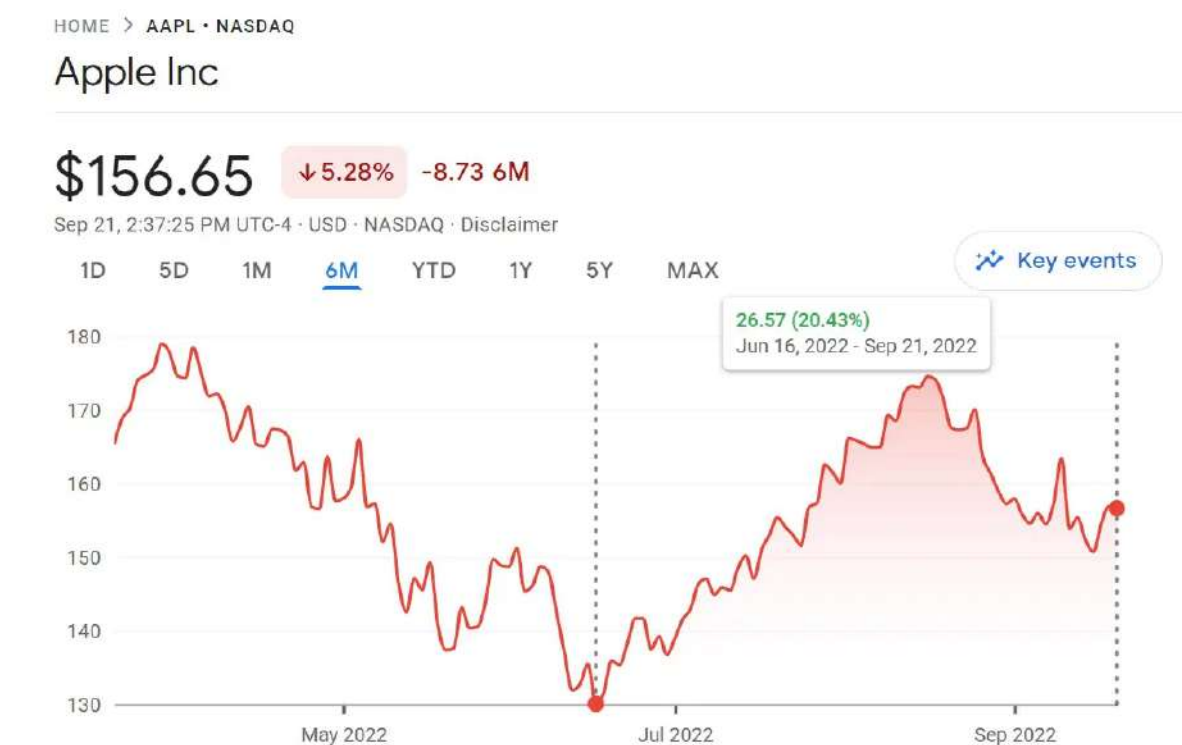
Language/text  
*sequences of words, tokens*



DNA sequence  
*sequences of nucleotide bases*



Audio/speech  
*sequences of waveform samples*



Finance  
*sequences of prices*

# Recap: Tokenization

| Tokenization type                                                                                    | Resulting tokens                                                                             |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Sentence-level                                                                                       | [ "I love this class!" ]                                                                     |
| Whitespace/word-level                                                                                | [ "I", "love", "this", "class!" ]                                                            |
| Word + punctuation                                                                                   | [ "I", "love", "this", "class", "!" ]                                                        |
| Character-level                                                                                      | [ "I", " ", "l", "o", "v", "e", " ", "t", "h", "i", "s", " ", "c", "l", "a", "s", "s", "!" ] |
| Subword-level                                                                                        | [ "I", "love", "the", "class", "!" ] or [ "I", "lov", "e", "the", "class", "!" ]             |
|  <b>Byte-level</b> | [ 73, 32, 108, 111, 118, 101, 32, 116, 104, 101, 32, 99, 108, 97, 115, 115, 33 ]             |

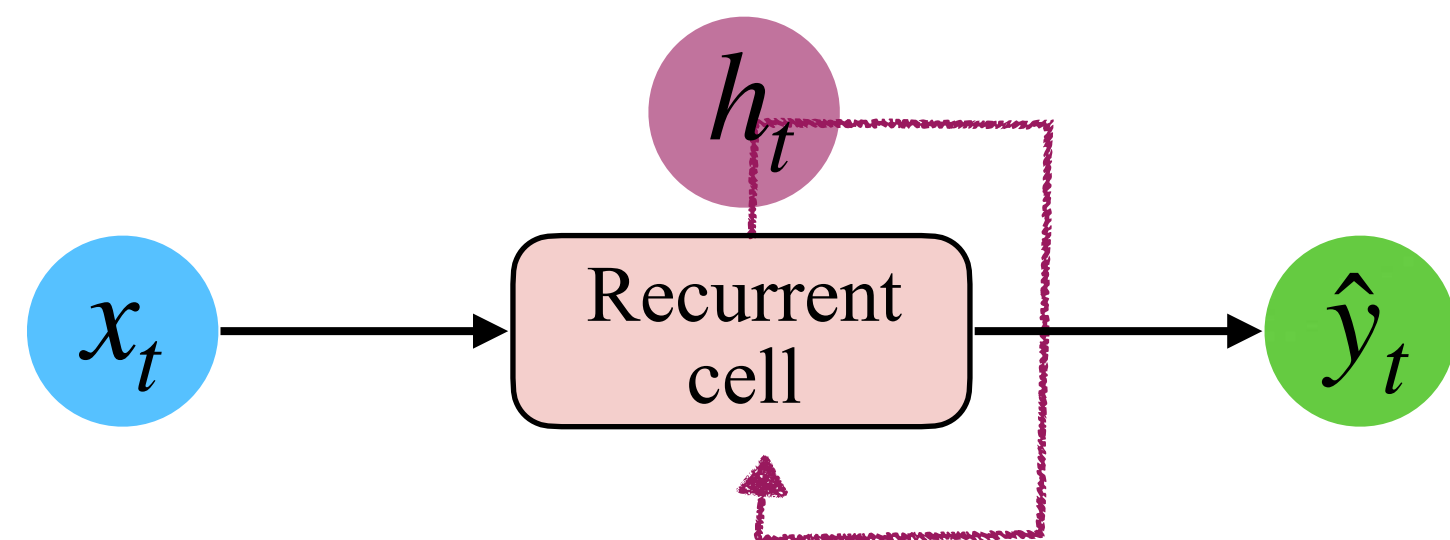
**Byte Pair Encoding (BPE):** Repeatedly merge the most frequent adjacent pair of tokens into a new token.

low      —> “low”  
lower    —> “low” “er”  
lowest   —> “low” “est”

# Recap: RNNs

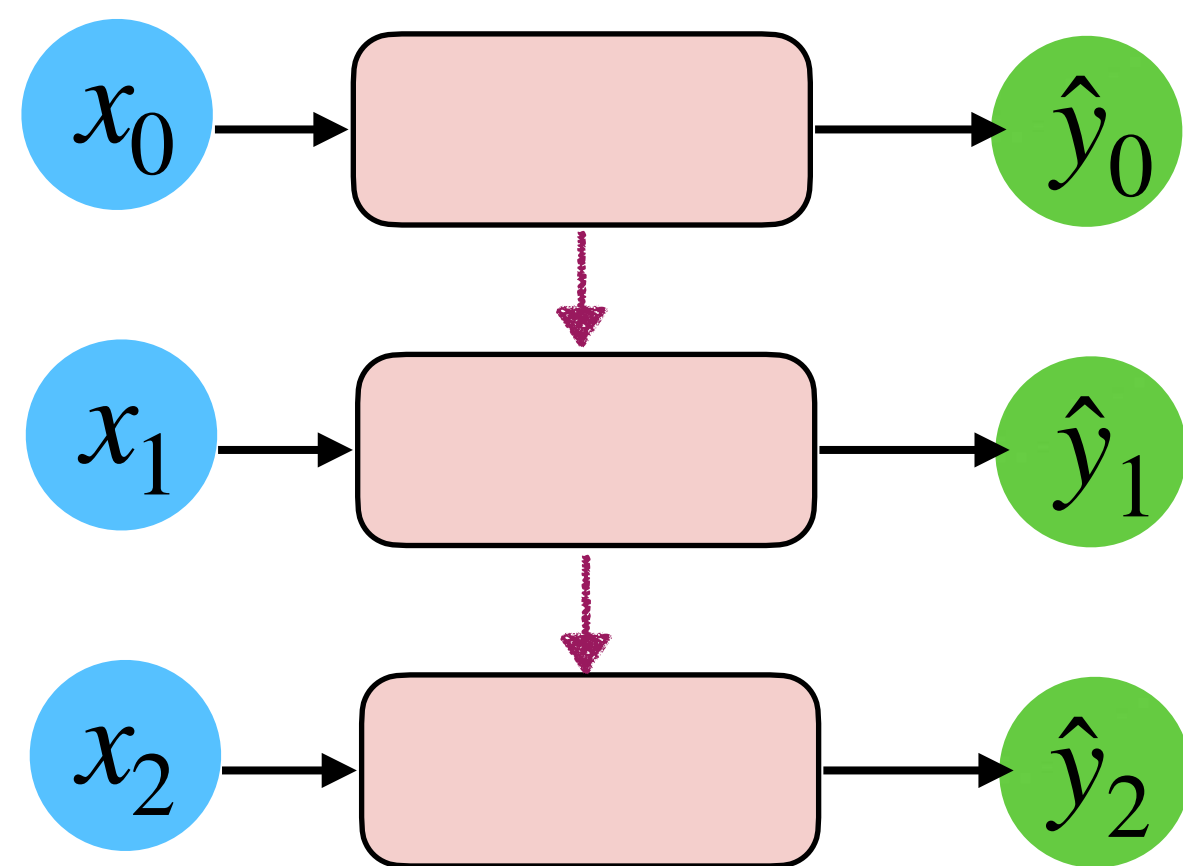


One to One “Vanilla” NN

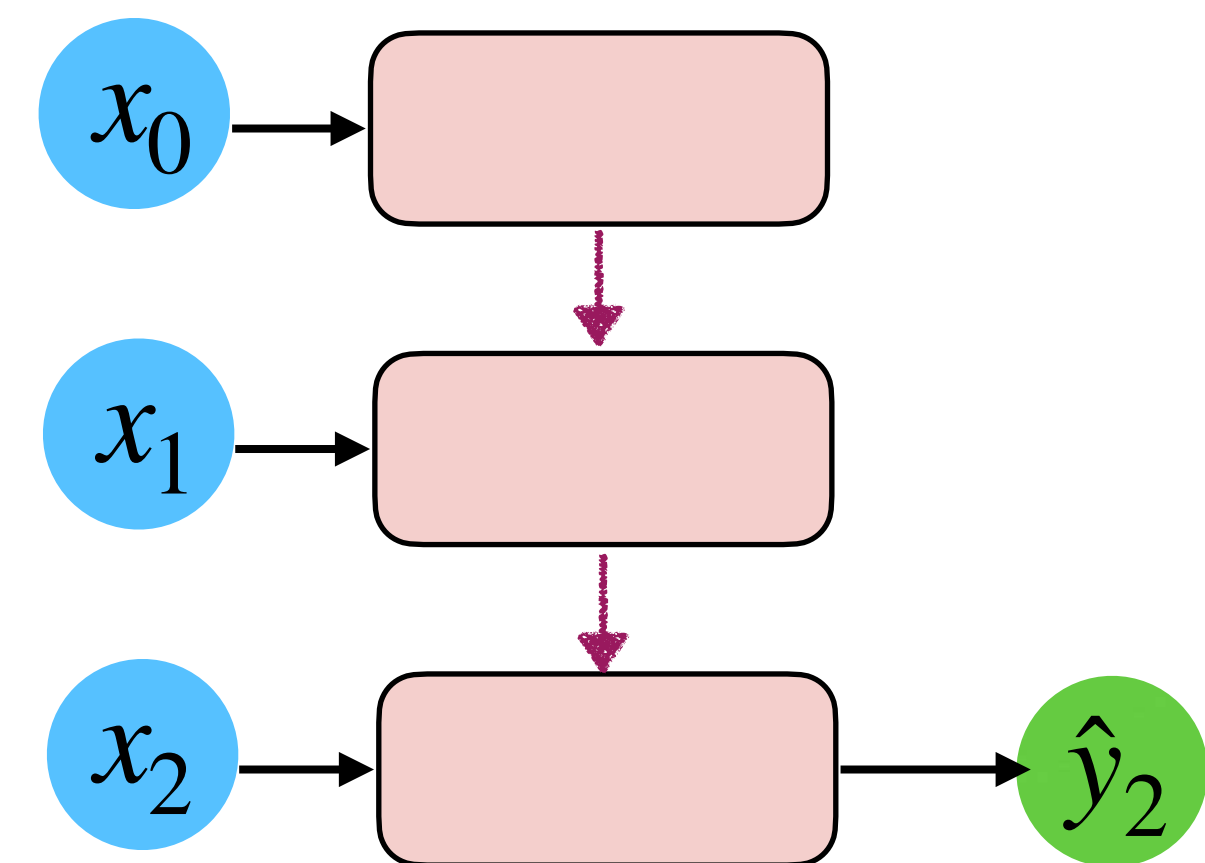


$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$

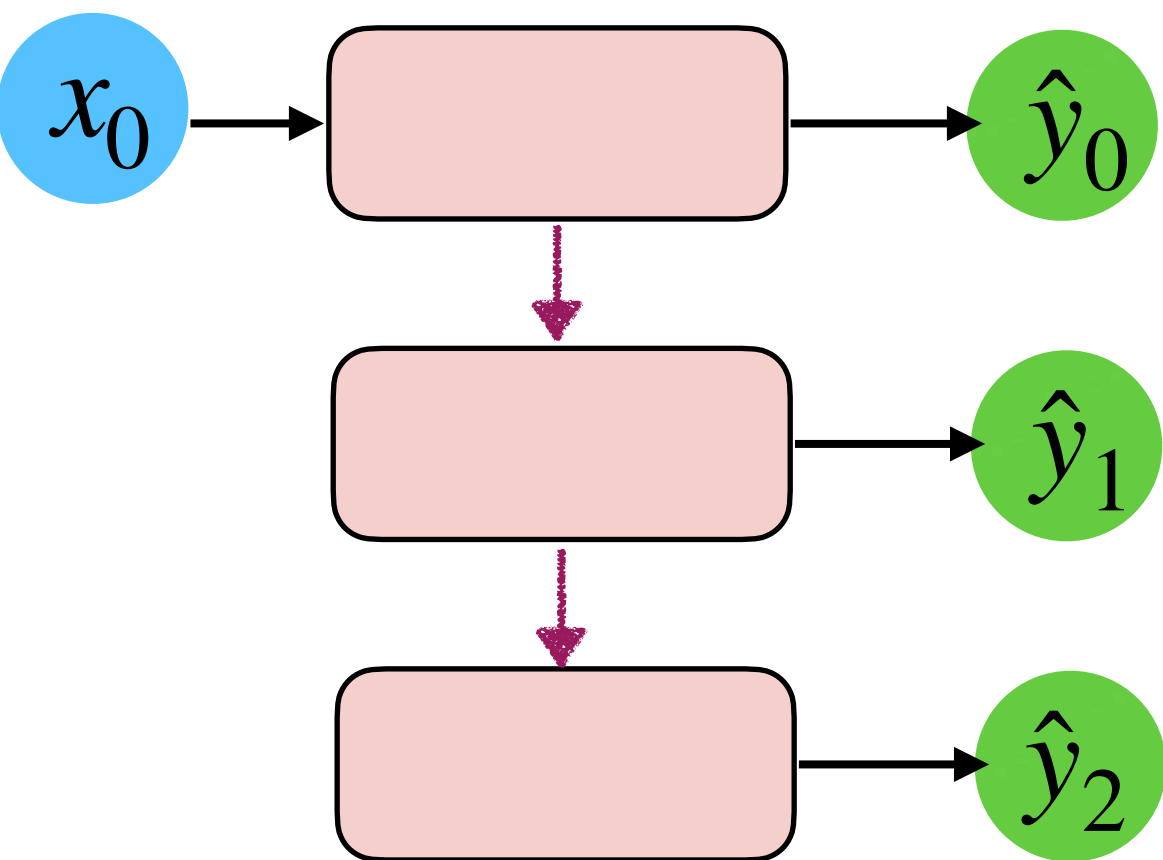
Output    Current state    Weight    Input    Last state



Many to Many  
Translation & Forecasting & Music Generation



Many to One  
Sentiment Classification

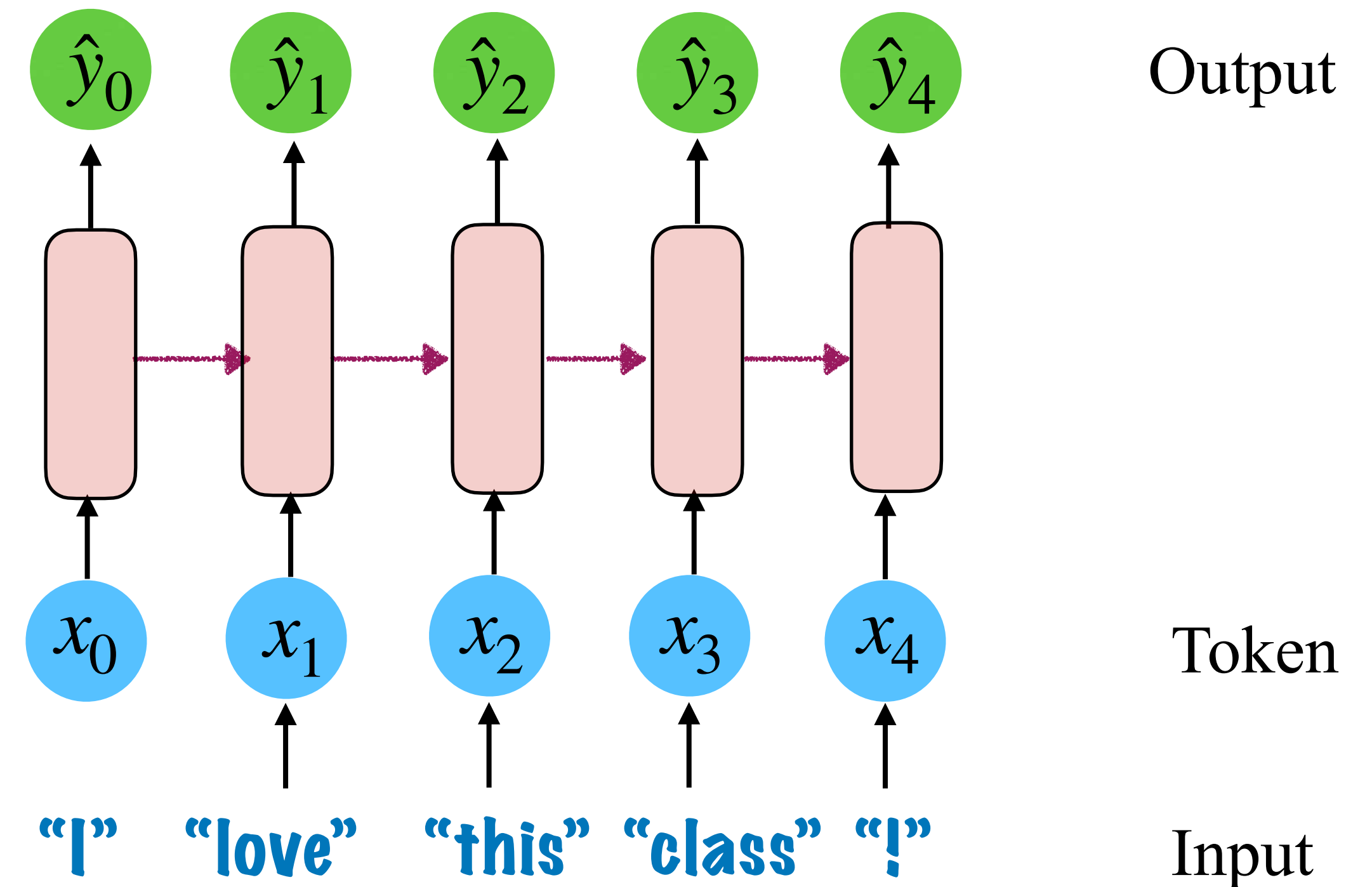


One to Many  
Text Generation & Video Generation

# Limitations of RNNs

To model sequences, we need to:

- ✓ 1. Handle **variable-length** sequences
- ✓ 2. **Share parameters** across the sequence
- ✓ 3. Maintain information about **order** / **Continuous stream**
- ✗ 4. ~~**Parallelization**~~ precessing
- ✗ 5. ~~Track **long-term** dependencies~~



# Parallelization —> fast training

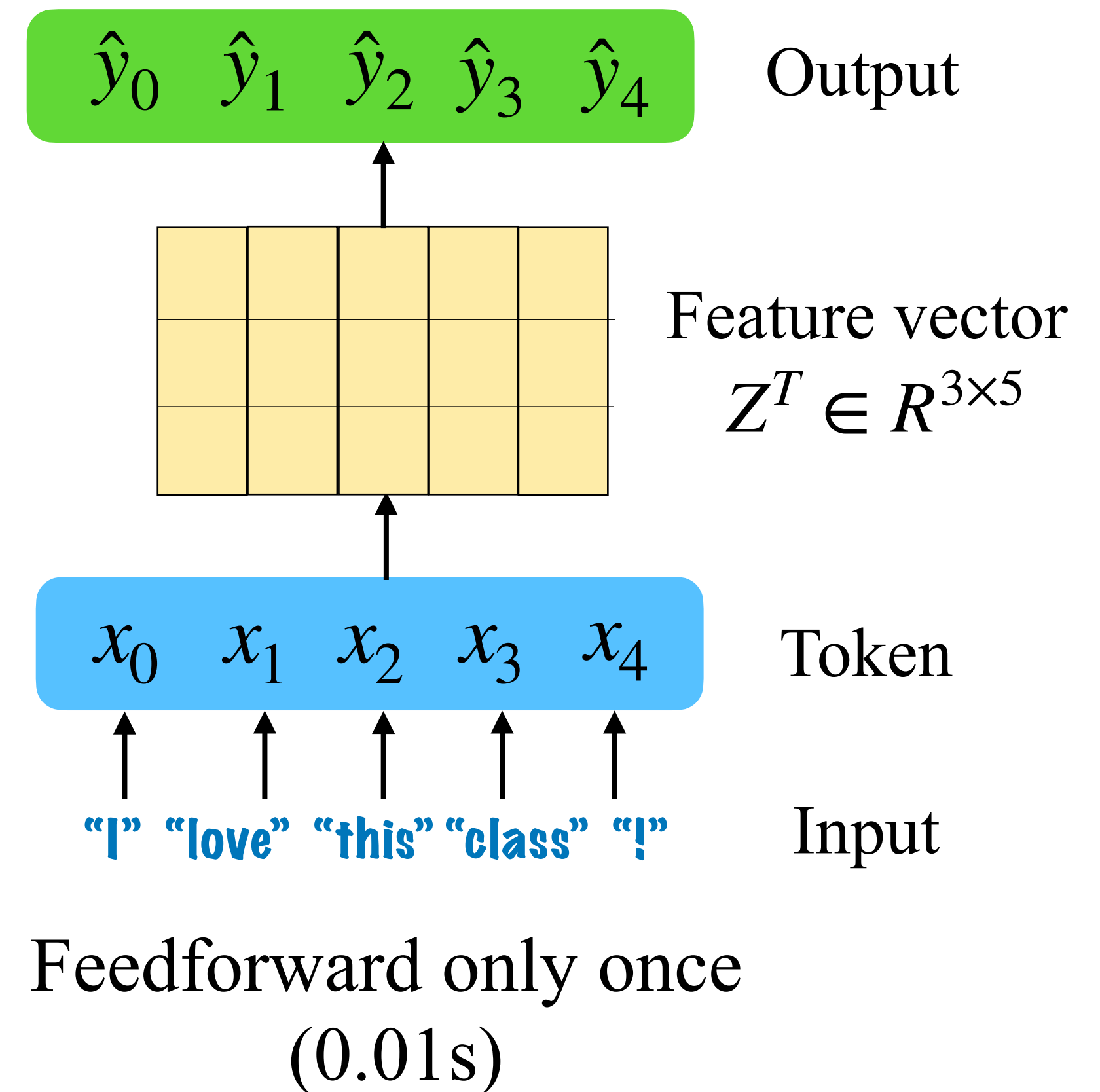
Core Idea: feed everything  
into dense network

$$[z_0, z_1, z_2, z_3, z_4] = W_{xh}[x_0, x_1, x_2, x_3, x_4]$$

Q1: How to encode the relationship  
between multiple tokens?

Q2: How to adapt to continuous  
stream?

(Only see previous tokens)



# Parallelization —> fast training:

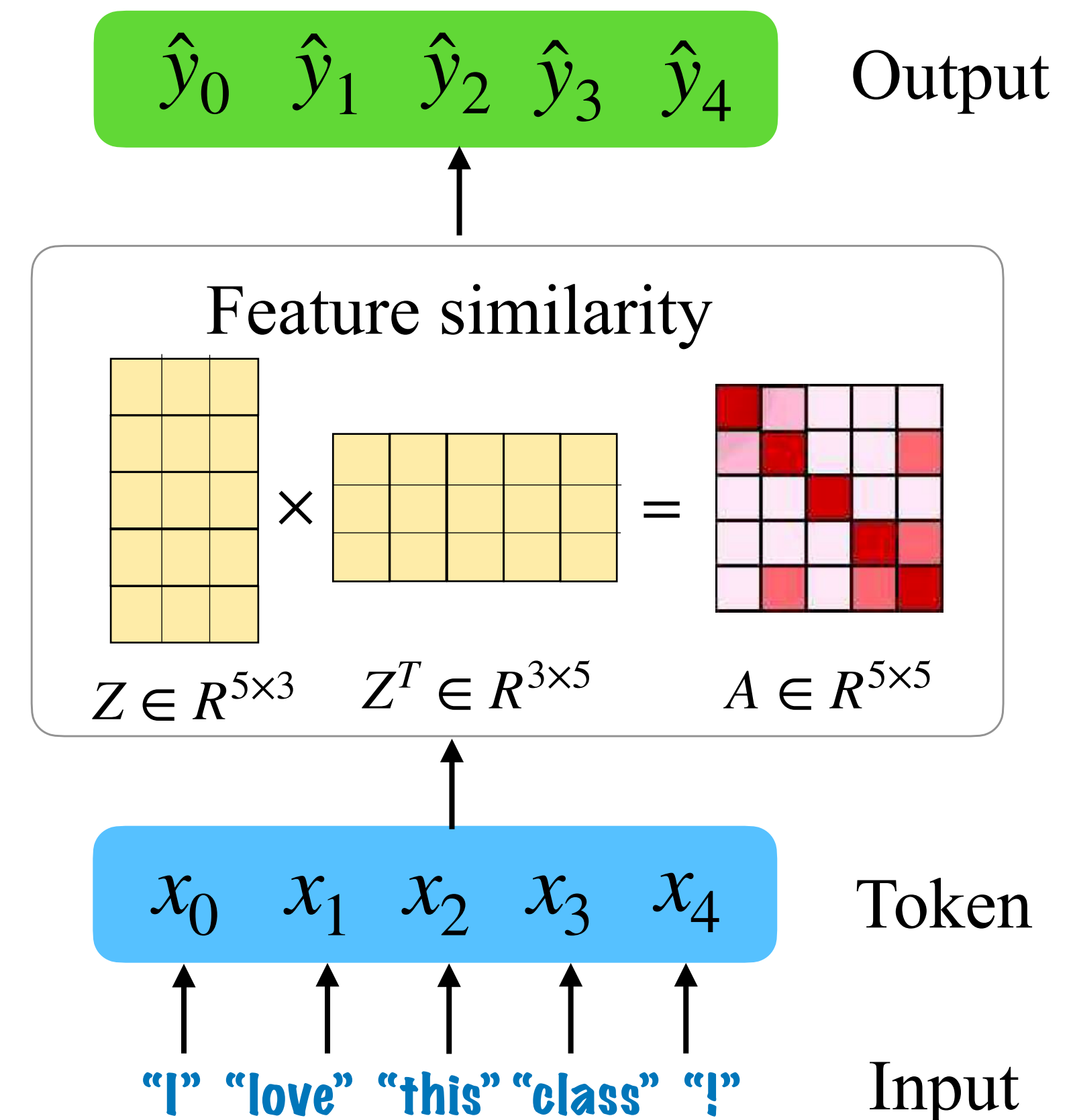
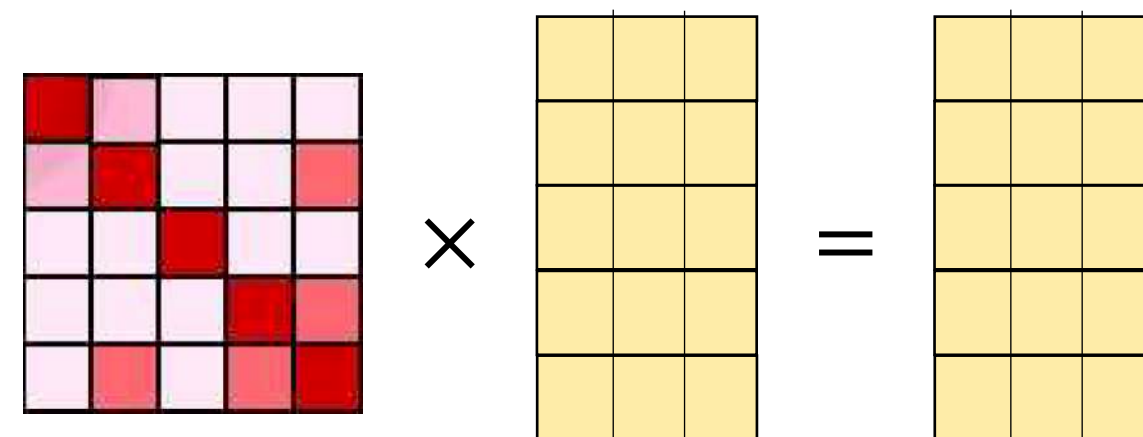
**Core Idea:** Feed everything into dense network

Q1: How to encode the relationship between multiple tokens?

Feature similarity:  $A = \text{Softmax}(ZZ^T) \in R^{5 \times 5}$

Feature update with similarity

$$Y = AZ \in R^{5 \times 3}$$



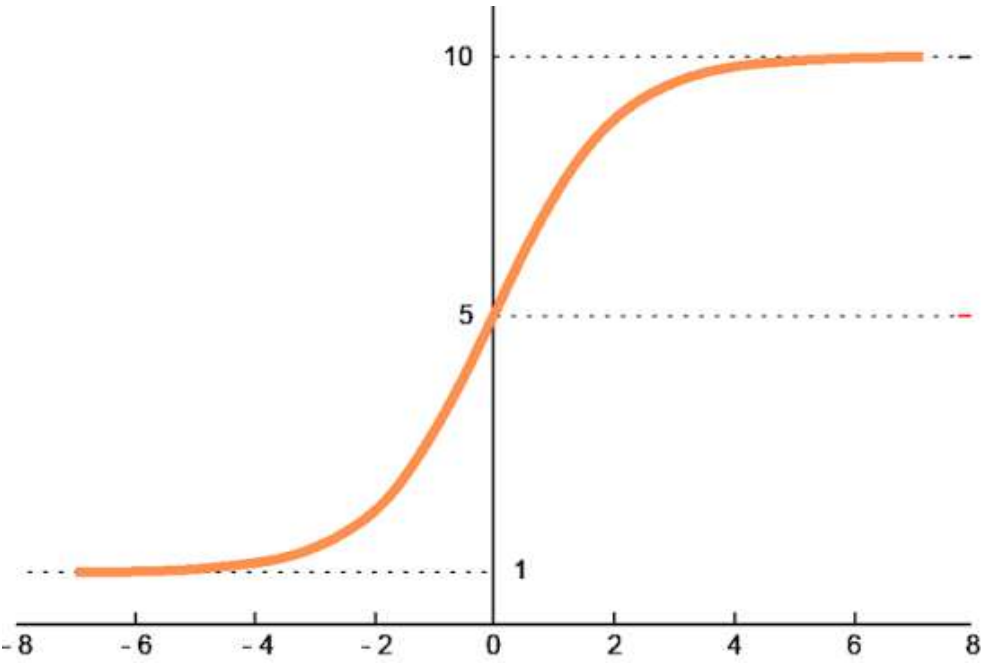
# Parallelization but not continuous stream:

## Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

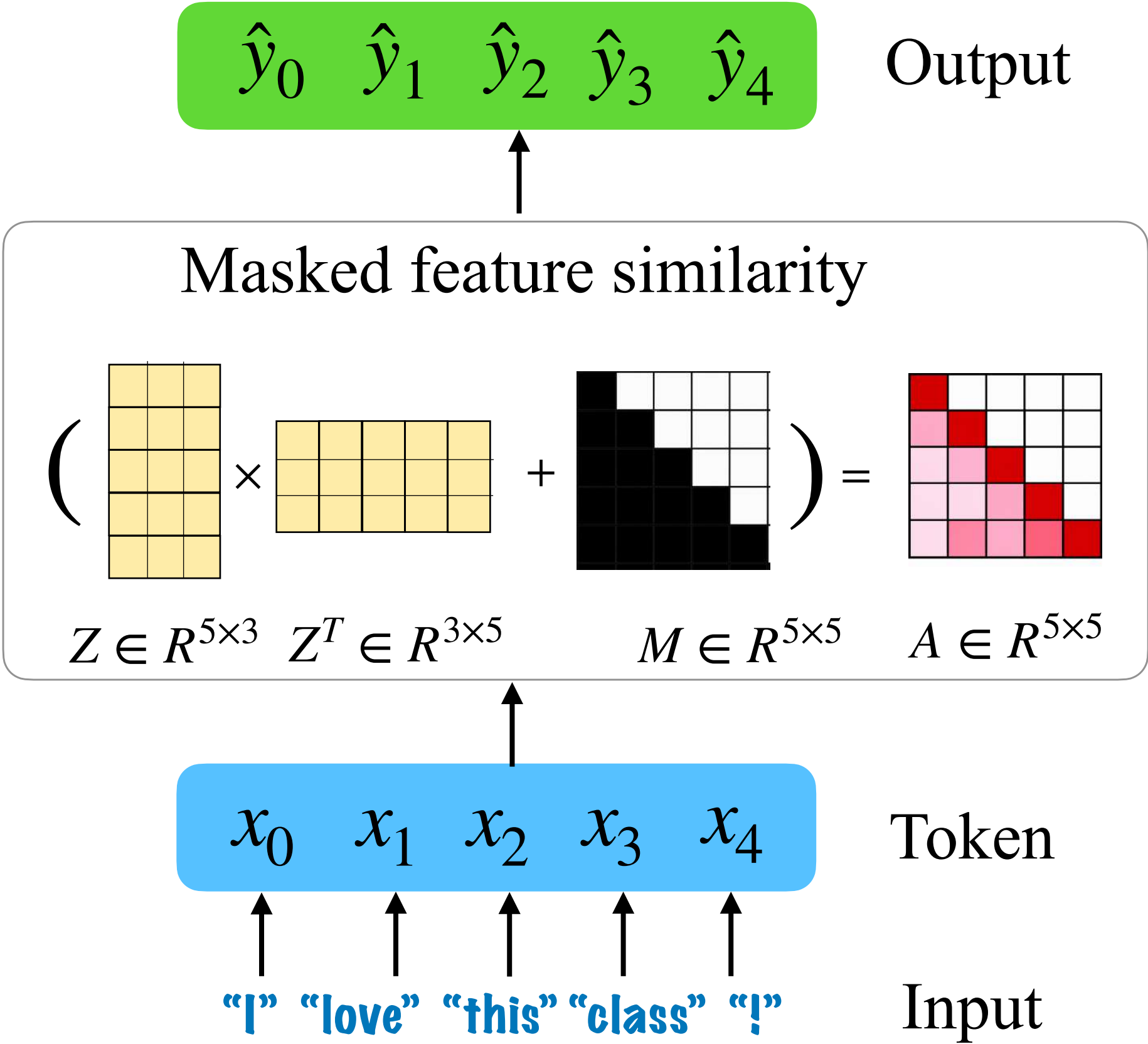
Only see previous tokens!

$$A = \text{softmax}(ZZ^T + M) \in R^{5 \times 5}$$



Softmax

|   |      |      |      |      |
|---|------|------|------|------|
| 0 | -inf | -inf | -inf | -inf |
| 0 | 0    | -inf | -inf | -inf |
| 0 | 0    | 0    | -inf | -inf |
| 0 | 0    | 0    | 0    | -inf |
| 0 | 0    | 0    | 0    | 0    |



# Parallelization $\Rightarrow$ Fast training:

Core Idea: Feed everything into dense network

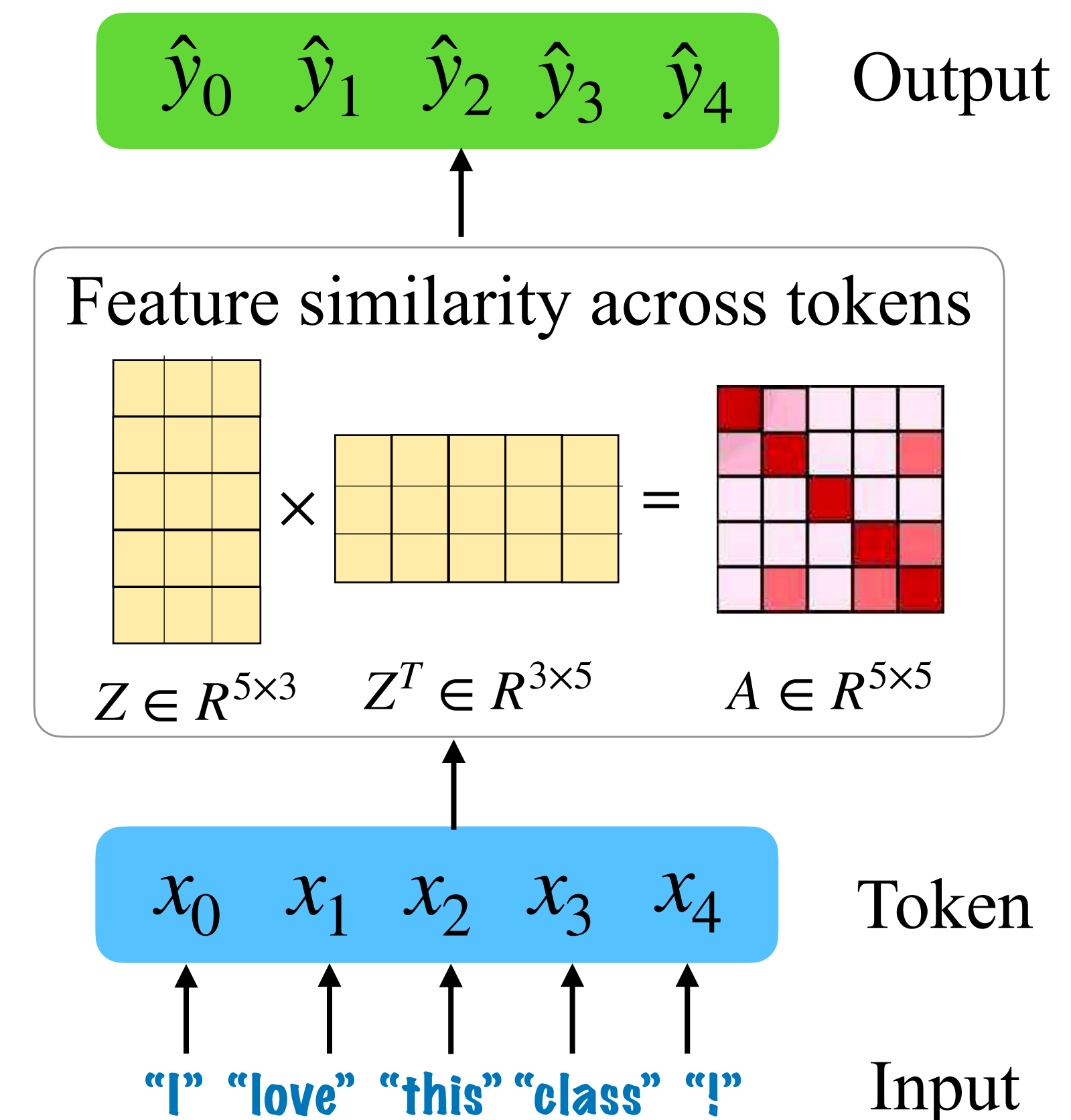
Q1: How to encode the relationship between multiple tokens?  
(Exchange information)

Feature similarity:

$$A = ZZ^T \in R^{5 \times 5}$$

|       | $z_0$ | $z_1$ | $z_2$ | $z_3$ | $z_4$ |
|-------|-------|-------|-------|-------|-------|
| $z_0$ | 150   | 89    | -25   | -36   | 56    |
| $z_1$ | 89    | 163   | -156  | -56   | 89    |
| $z_2$ | -25   | -156  | 452   | 26    | 54    |
| $z_3$ | -36   | -56   | 26    | 562   | 362   |
| $z_4$ | 56    | 89    | 54    | 362   | 268   |

dot-product similarity between two feature vectors



# Goal of Sequence Modeling

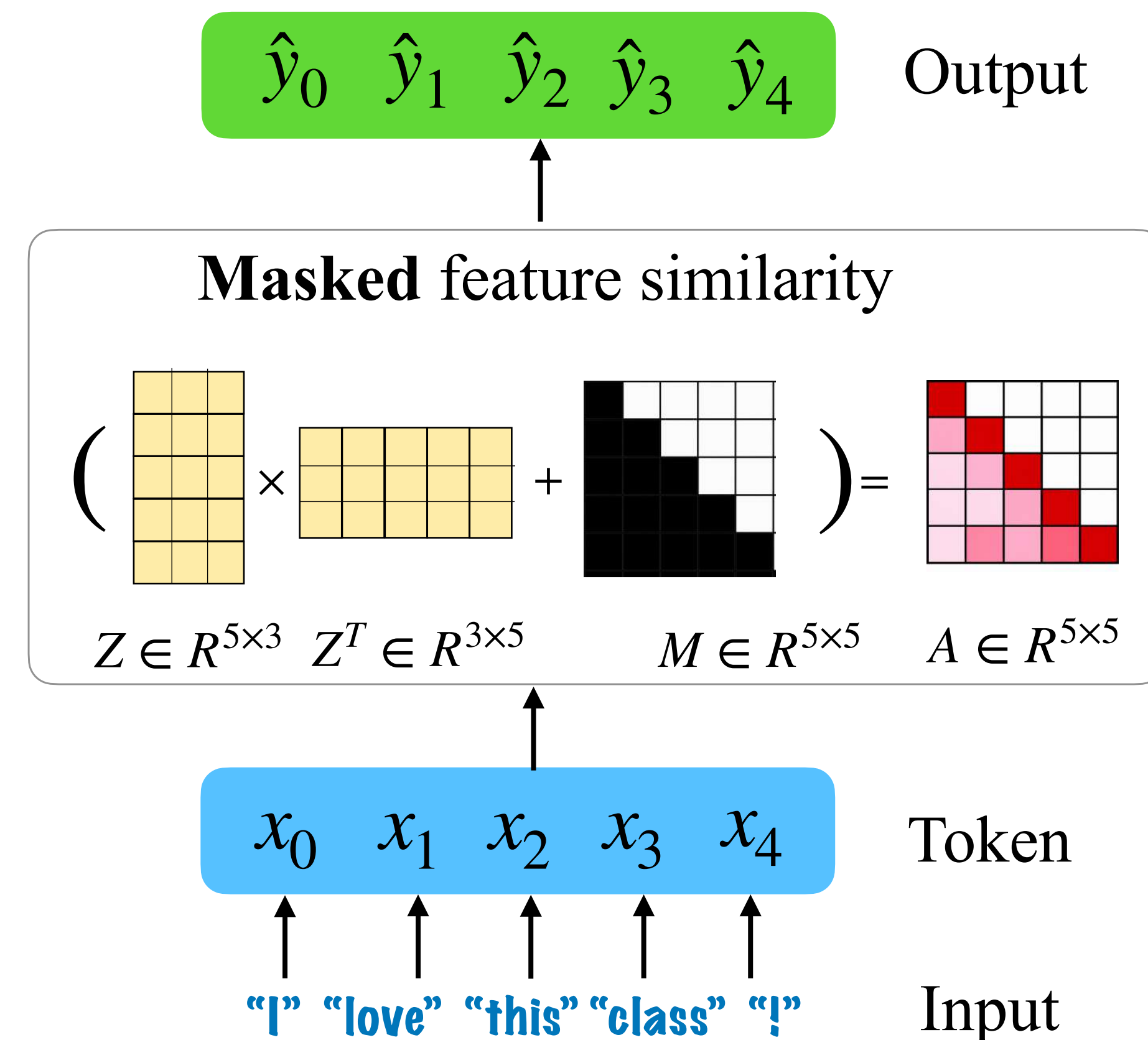
**Core Idea:** Feed everything into dense network

## Desired Capabilities:

- ✓ No recurrence
- ✓ Continuous stream (online stream)
- ✓ Parallelization
- ✗ Long-term memory (Mamba Model)

Causal attention

KV Cache



# Attention Is All You Need

- Scaled Dot-Product Attention
- Multi-head Attention
- Self-attention v.s. cross-attention
- Positional Embeddings (PE)
- Feed Forward Networks (FFNs)

Paper: [/https://arxiv.org/pdf/1706.03762](https://arxiv.org/pdf/1706.03762)

Visualization Github Repo: <https://github.com/jessevig/bertviz>

# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$

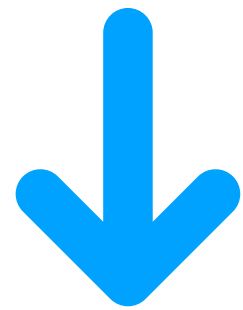
Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$



Remove  
mask

$$= \text{softmax}(ZZ^T) Z$$

Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

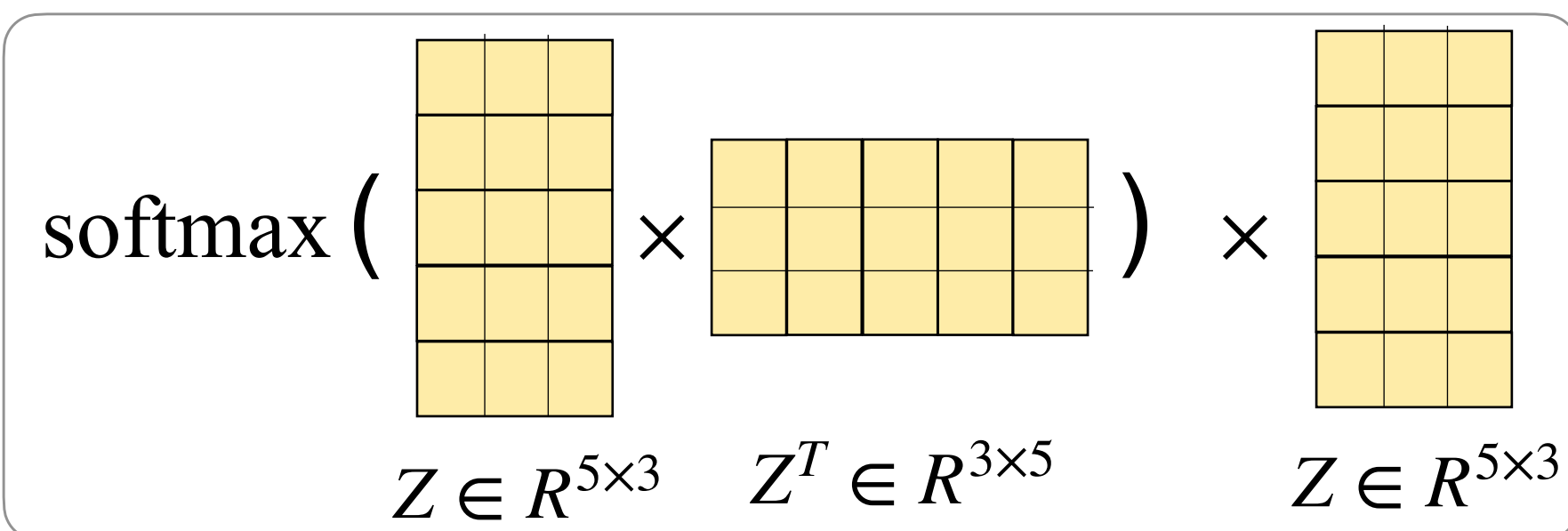
# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$



$$= \text{softmax}(ZZ^T) Z$$



Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$



$$= \text{softmax}(ZZ^T) Z$$

$\text{softmax} \left( \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \times \begin{array}{|c|c|c|c|c|} \hline \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square \\ \hline \end{array} \right) \times \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

$Z \in R^{5 \times 3} \quad Z^T \in R^{3 \times 5} \quad Z \in R^{5 \times 3}$

Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

**It is good. But what are main limitations?**

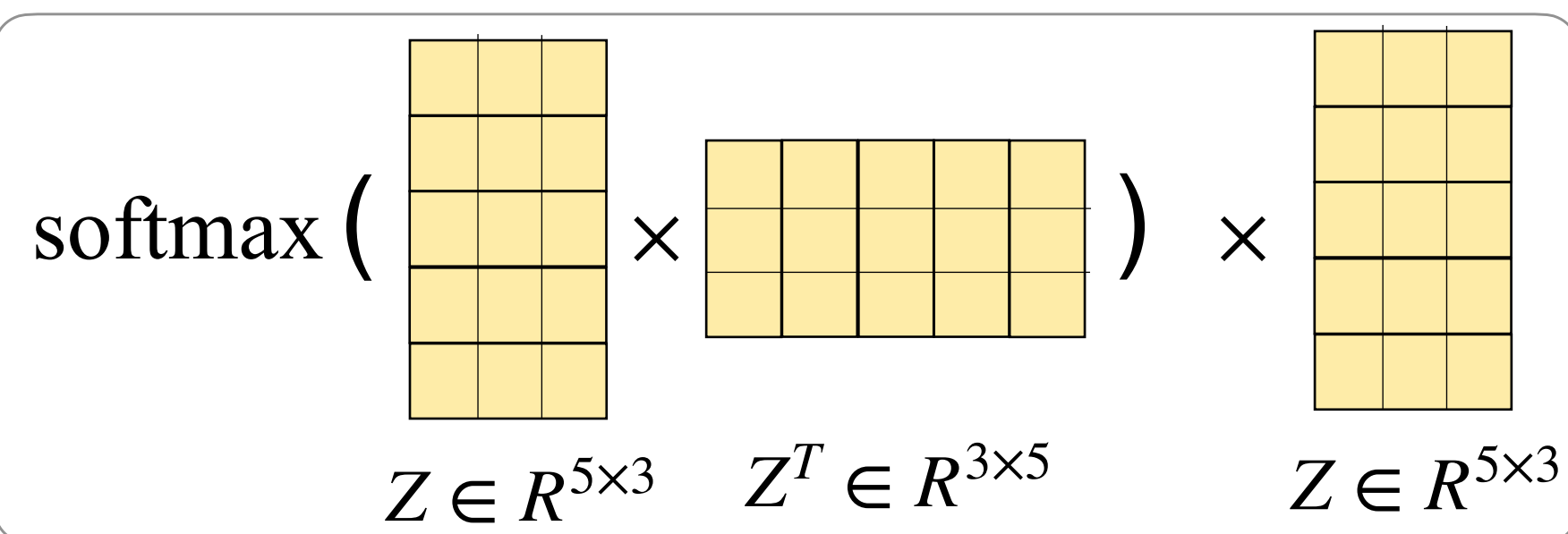
# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$



$$= \text{softmax}(ZZ^T) Z$$



$\text{softmax} \left( \begin{matrix} \text{5x3} \\ \times \\ \text{3x5} \end{matrix} \right) \times \begin{matrix} \text{5x5} \\ \text{5x3} \end{matrix}$

$Z \in R^{5 \times 3} \quad Z^T \in R^{3 \times 5} \quad Z \in R^{5 \times 3}$

Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

**It is good. But what are main limitations?**

- **same space for matching and retrieval**
- **symmetric pre-softmax scores:  $a_{ij} = a_{ji}$**
- **limited diversity**

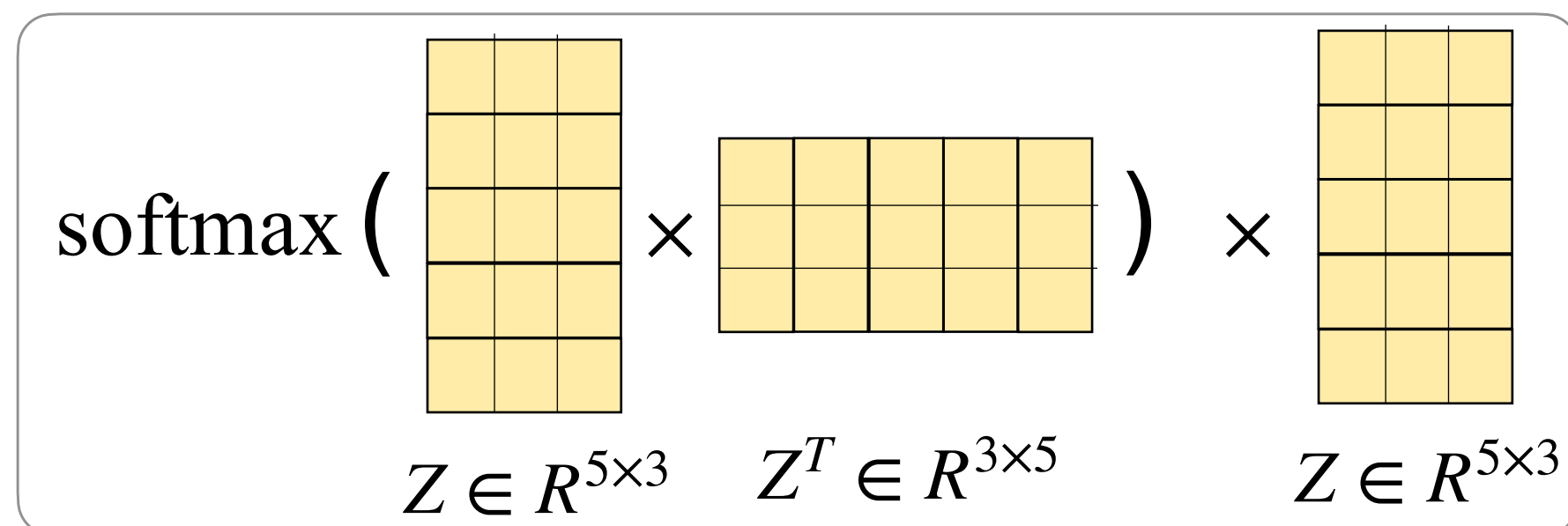
# Scaled Dot-Product Attention

Feature update with similarity and mask

$$\begin{aligned} Y &= \text{softmax}(A \odot M) Z \\ &= \text{softmax}((ZZ^T) \odot M) Z \end{aligned}$$



$$= \text{softmax}(ZZ^T) Z$$



$\text{softmax} \left( \begin{matrix} \text{5x3} & \times & \text{3x5} \end{matrix} \right) \times \begin{matrix} \text{5x3} \end{matrix}$

$Z \in R^{5 \times 3} \quad Z^T \in R^{3 \times 5} \quad Z \in R^{5 \times 3}$

Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Introducing projection layers

$$Z \xrightarrow{W_q} Q \quad Z \xrightarrow{W_k} K \quad Z \xrightarrow{W_v} V$$

# Attention: Query, Key & Value

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_q \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \begin{array}{c} Q \\ \text{Query} \end{array}$$

Introducing projection layers

$$Z \xrightarrow{W_q} Q \quad Z \xrightarrow{W_k} K \quad Z \xrightarrow{W_v} V$$

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_k \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \begin{array}{c} K \\ \text{Key} \end{array}$$

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_v \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \begin{array}{c} V \\ \text{Value} \end{array}$$

# Attention: Query, Key & Value

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_q \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \begin{array}{c} Q \\ \text{Query} \end{array}$$

Attention formulation:

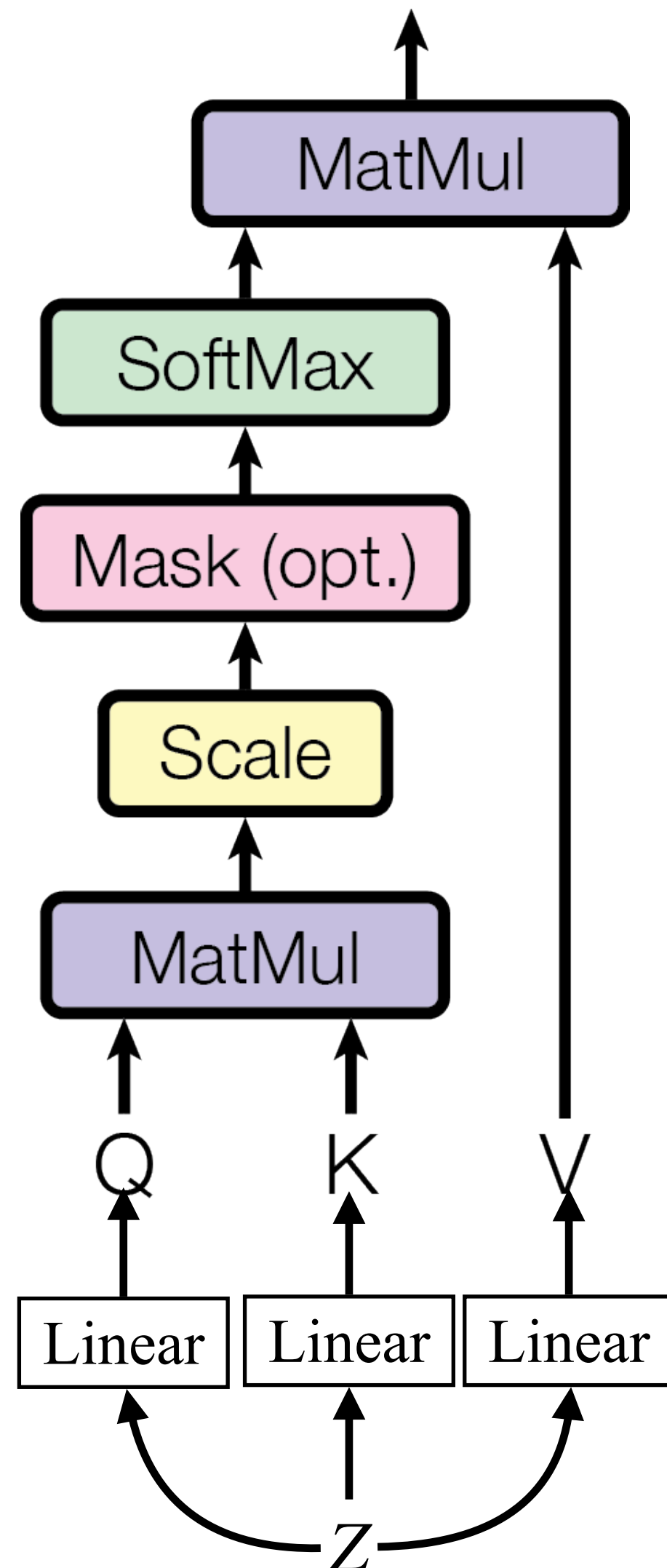
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_k \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \begin{array}{c} K \\ \text{Key} \end{array}$$

$$Z \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{c} W_v \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{array} = \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \begin{array}{c} V \\ \text{Value} \end{array}$$

$$\text{softmax}\left(\frac{\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array}}{\sqrt{d_k}}\right) \times \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array}$$

# Scaled Dot-Product Attention



Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The diagram shows the matrix operations for the attention formulation. It features three matrices:  $Q \in R^{5 \times d_k}$  (blue, 5 rows by 4 columns),  $K^T \in R^{d_k \times 5}$  (orange, 4 rows by 5 columns), and  $V \in R^{5 \times d_v}$  (purple, 5 rows by 4 columns). The equation is represented as:

$$\text{softmax}\left(\frac{\begin{matrix} \text{blue matrix} \\ \times \\ \text{orange matrix} \end{matrix}}{\sqrt{d_k}}\right) \times \begin{matrix} \text{purple matrix} \end{matrix}$$

# Attention: Dot-Product?

- Scaled dot-product

$$a_{ij} = \frac{q_i k_j^T}{\sqrt{d_k}}$$

- Scaled cosine attention

$$a_{ij} = \frac{1}{\tau} \frac{q_i k_j^T}{\|q_i\|_2 \|k_j\|_2}$$

- Kernel attention

$$a_{ij} = \phi(q_i) \phi(k_j)^T$$

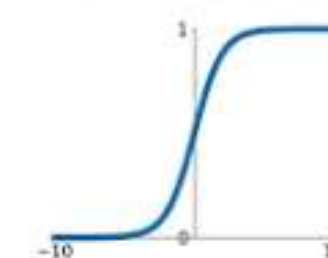


$$Y = \phi(Q)(\phi(K)^T V) \quad \text{Linear Attention}$$

## Activation Functions

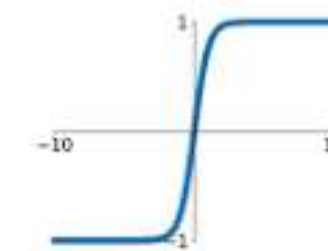
**Sigmoid**

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



**tanh**

$$\tanh(x)$$



**ReLU**

$$\max(0, x)$$



**Leaky ReLU**

$$\max(0.1x, x)$$

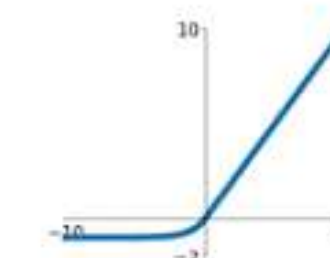


**Maxout**

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

**ELU**

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



$$\phi(x) = \text{ELU}(x) + 1$$

Exponential Linear Unit

# Attention Is All You Need

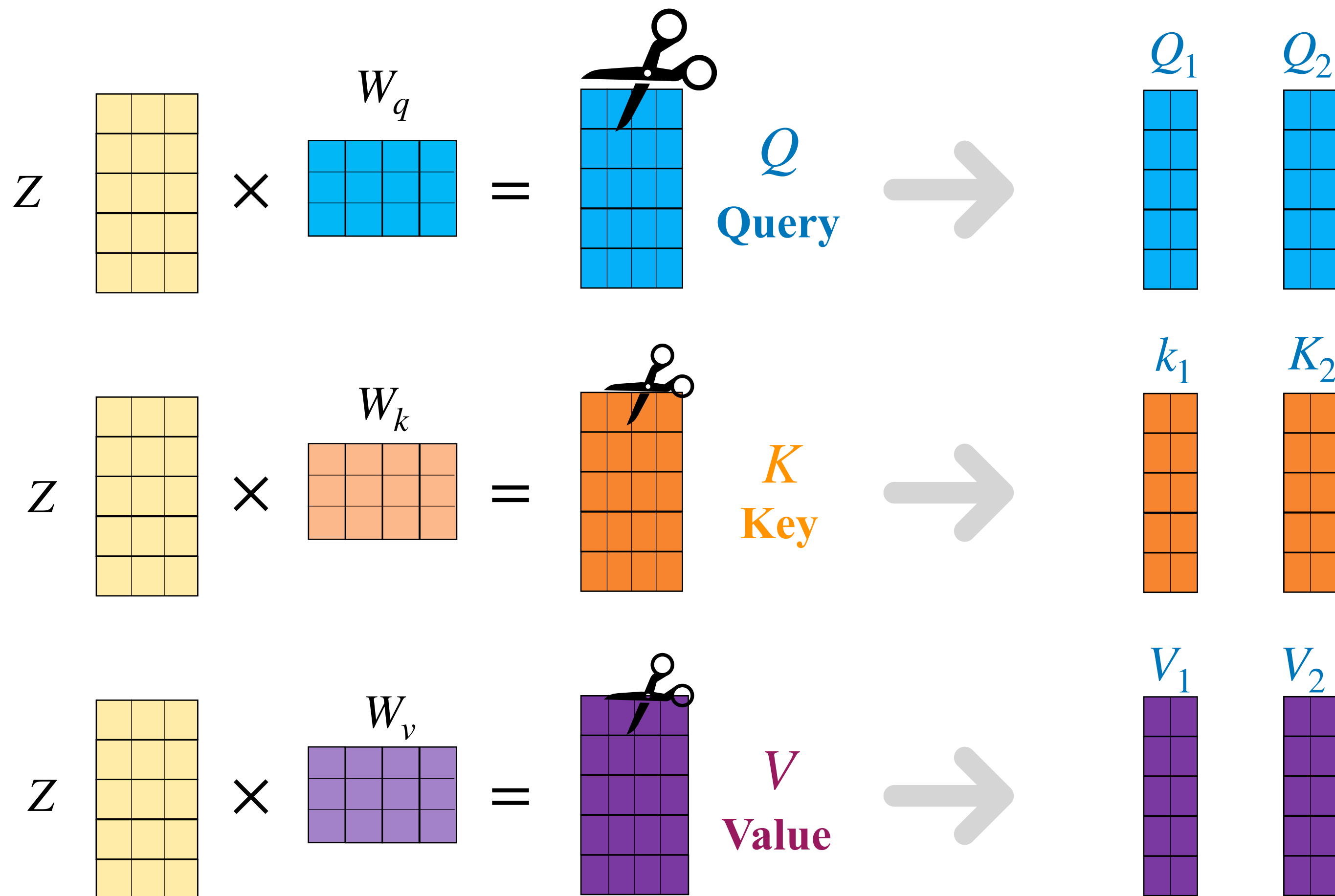
- Scaled Dot-Product Attention
- Multi-head Attention
- Feed Forward Networks (FFNs)
- Positional Embeddings (PE)
- Self-attention v.s. cross-attention

Paper: [/https://arxiv.org/pdf/1706.03762](https://arxiv.org/pdf/1706.03762)

Visualization Github Repo: <https://github.com/jessevig/bertviz>

# Multi-Head Attention

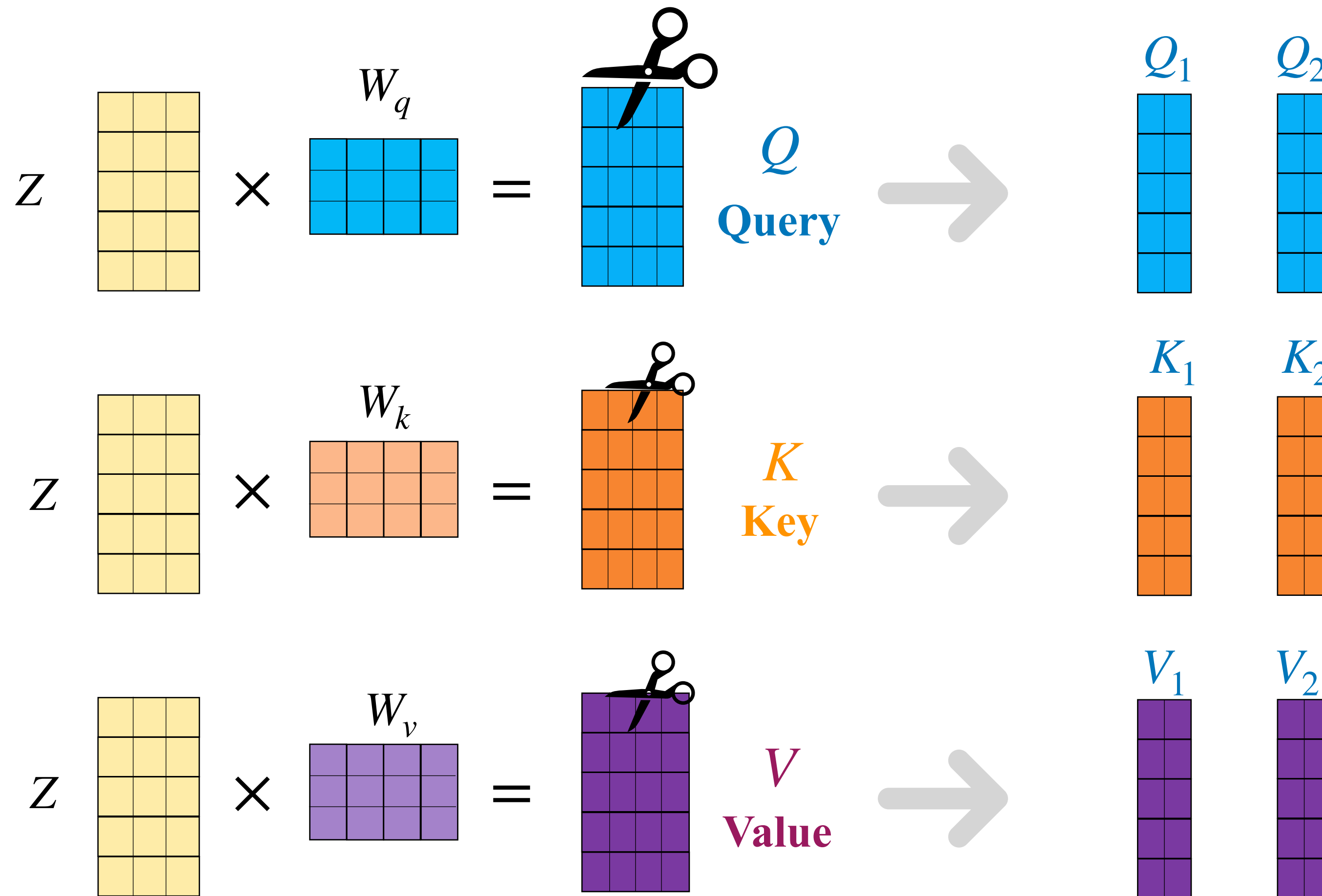
“I” “love” “this” “class” “!”



Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions

# Multi-Head Attention

“I” “love” “this” “class” “!”



Head / Subspace 1

$$Q_1 \in R^{5 \times \frac{d_k}{2}} \quad K_1^T \in R^{\frac{d_k}{2} \times 5} \quad V_1 \in R^{5 \times \frac{d_v}{2}}$$
$$\text{softmax} \left( \frac{\begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline \text{orange} \\ \hline \end{array}}{\sqrt{d_k}} \right) \times \begin{array}{|c|} \hline \text{purple} \\ \hline \end{array}$$

Head / Subspace 2

$$Q_2 \in R^{5 \times \frac{d_k}{2}} \quad K_2^T \in R^{\frac{d_k}{2} \times 5} \quad V_2 \in R^{5 \times \frac{d_v}{2}}$$
$$\text{softmax} \left( \frac{\begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline \text{orange} \\ \hline \end{array}}{\sqrt{d_k}} \right) \times \begin{array}{|c|} \hline \text{purple} \\ \hline \end{array}$$

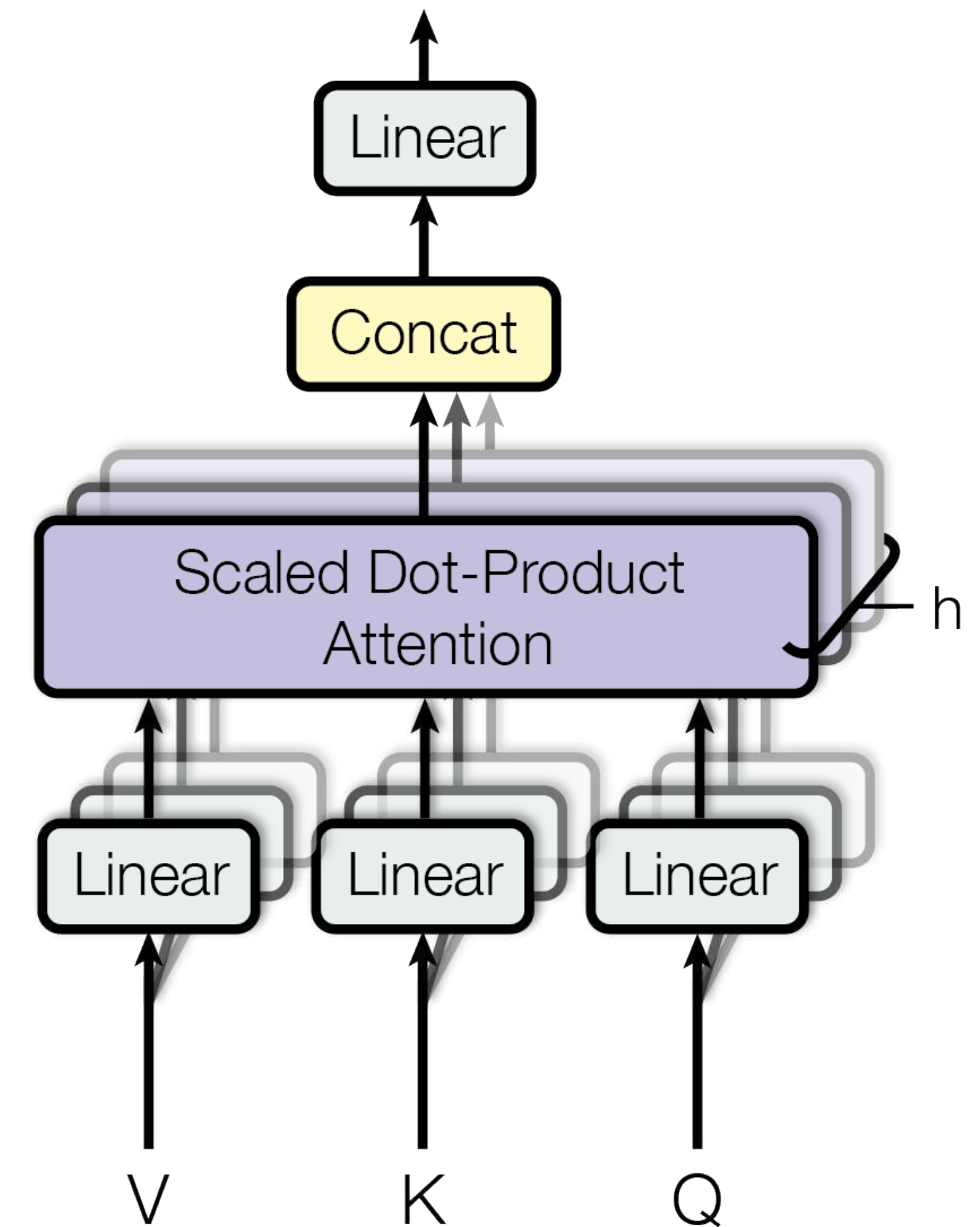
# Multi-Head Attention

Multi-Head Attention formulation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^{\top}}{\sqrt{d_k}} \right) V$$

$$\text{head}_h = \text{Attention} \left( QW_h^Q, KW_h^K, VW_h^V \right)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat} (\text{head}_1, \dots, \text{head}_H) W^O$$



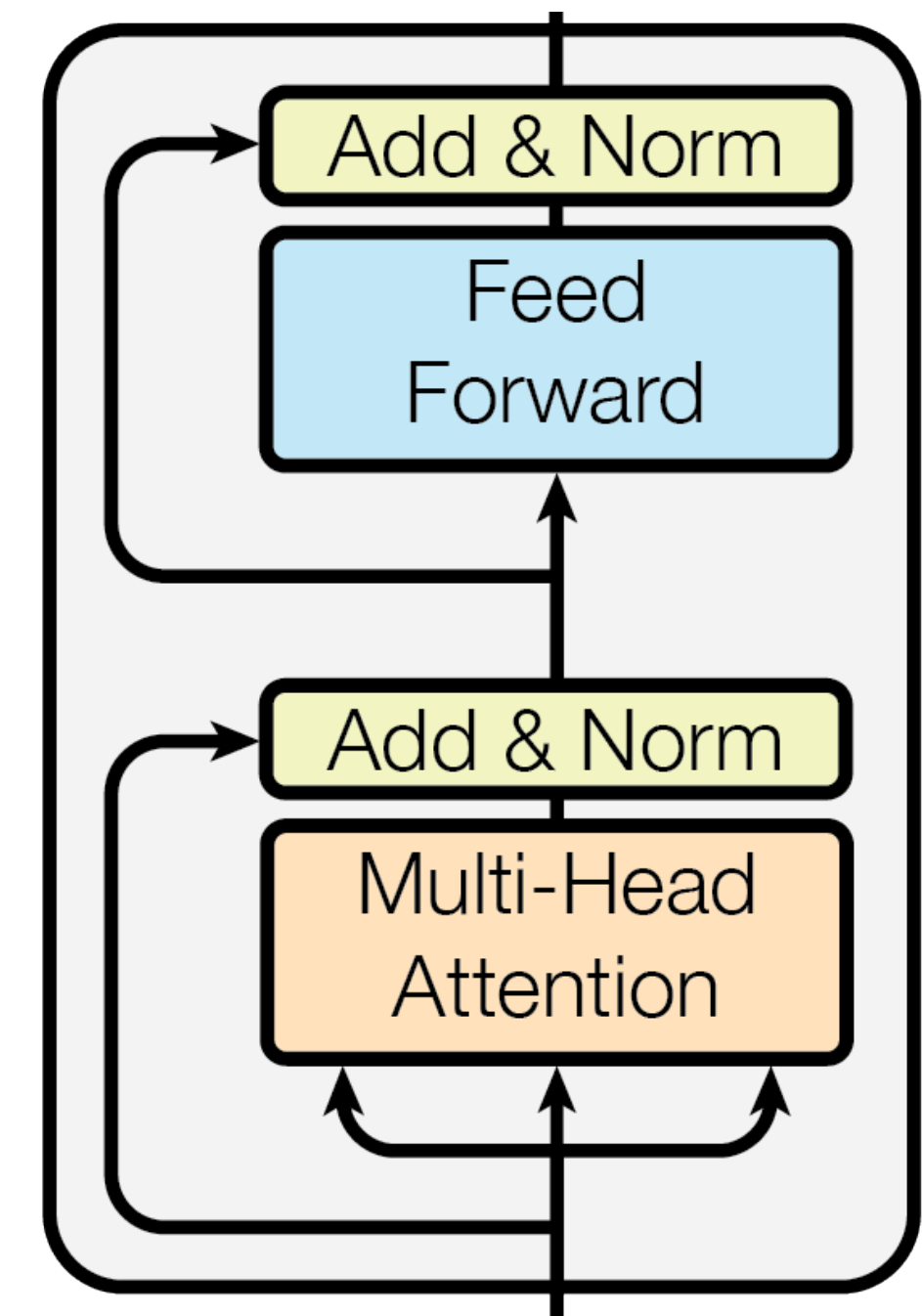
# Feed-Forward Networks (FFNs)

a fully connected  
Feed Forward Network

Two-layer MLP

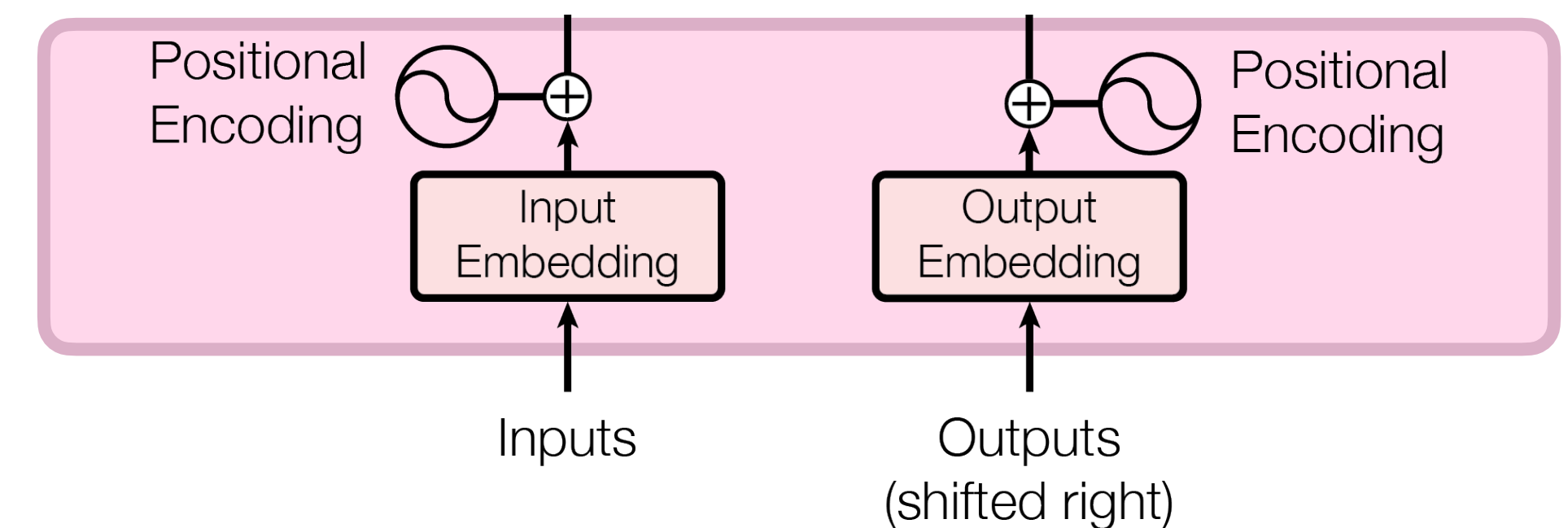
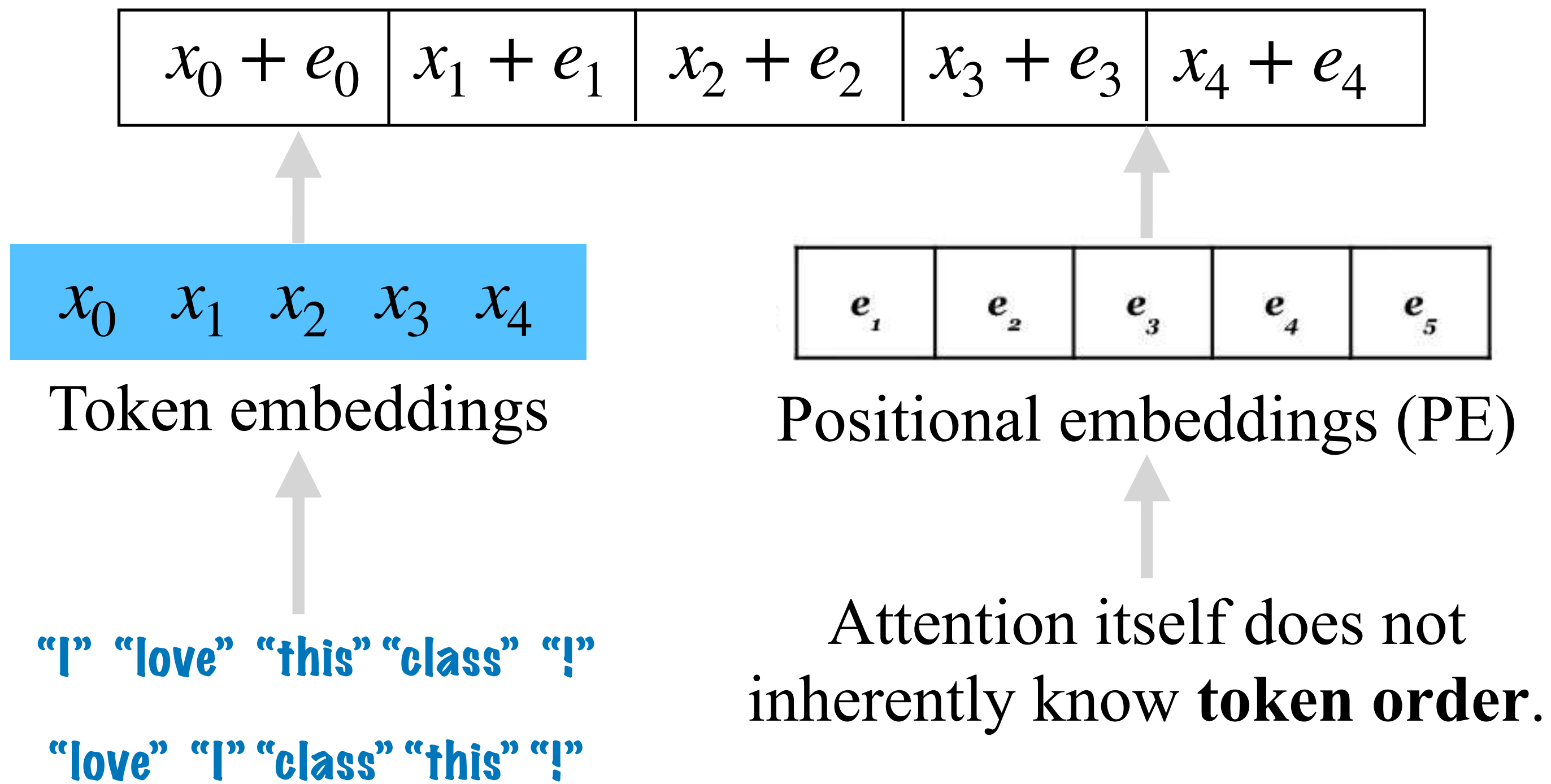
$$\text{FFN}(x) = \text{RELU} \left( x \underset{512 \times 2048}{W_1} + b_1 \right) \underset{2048 \times 512}{W_2} + b_2$$

$d \rightarrow 4d \rightarrow d$



A single transformer layer

# Positional Embeddings (PEs)



# Sinusoidal Positional Embeddings

Different dimensions use different frequencies.

Formulation for Positional Encoding

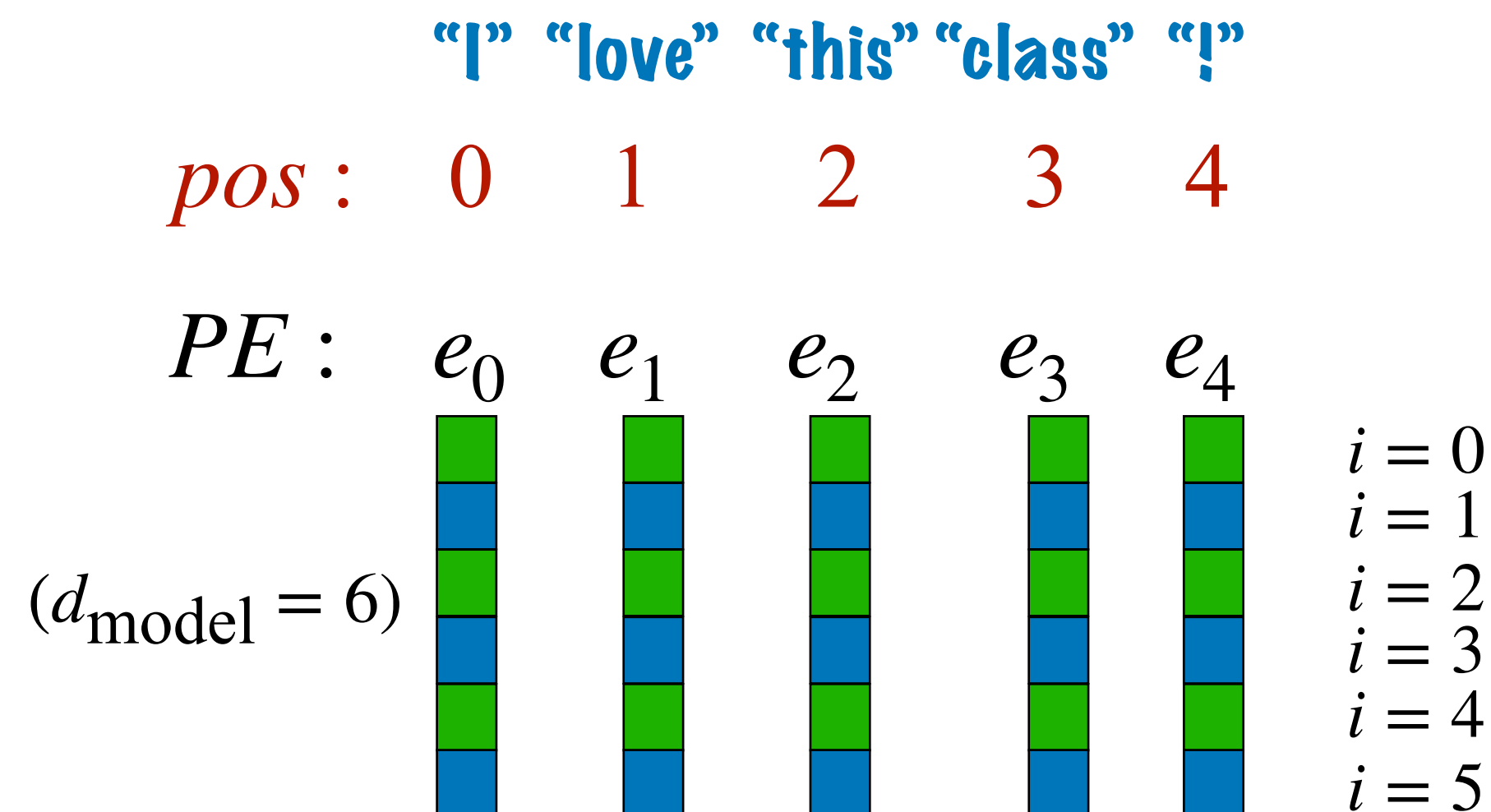
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

$pos$  = token position index in the sequence

$i$  = embedding dimension index

$d_{\text{model}}$  = embedding dimension

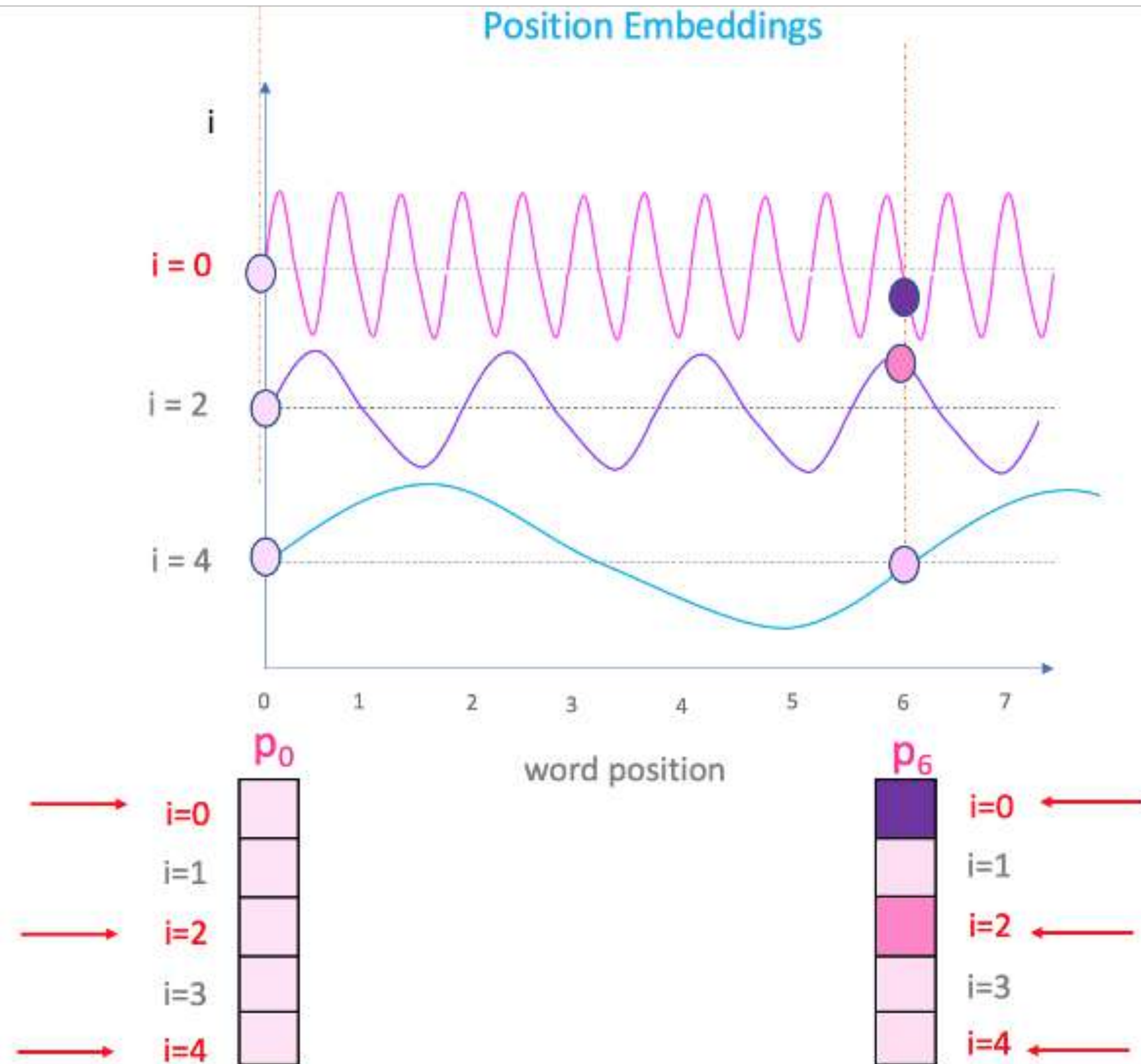


$$e_0 = PE(pos = 0, *) =$$

$$\begin{bmatrix} \sin(0/10000^{(2 \times 0)/6}) \\ \cos(0/10000^{(2 \times 0)/6}) \\ \sin(0/10000^{(2 \times 1)/6}) \\ \cos(0/10000^{(2 \times 1)/6}) \\ \sin(0/10000^{(2 \times 2)/6}) \\ \cos(0/10000^{(2 \times 2)/6}) \end{bmatrix}$$

# Transformer: Sinusoidal Positional Embeddings

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$



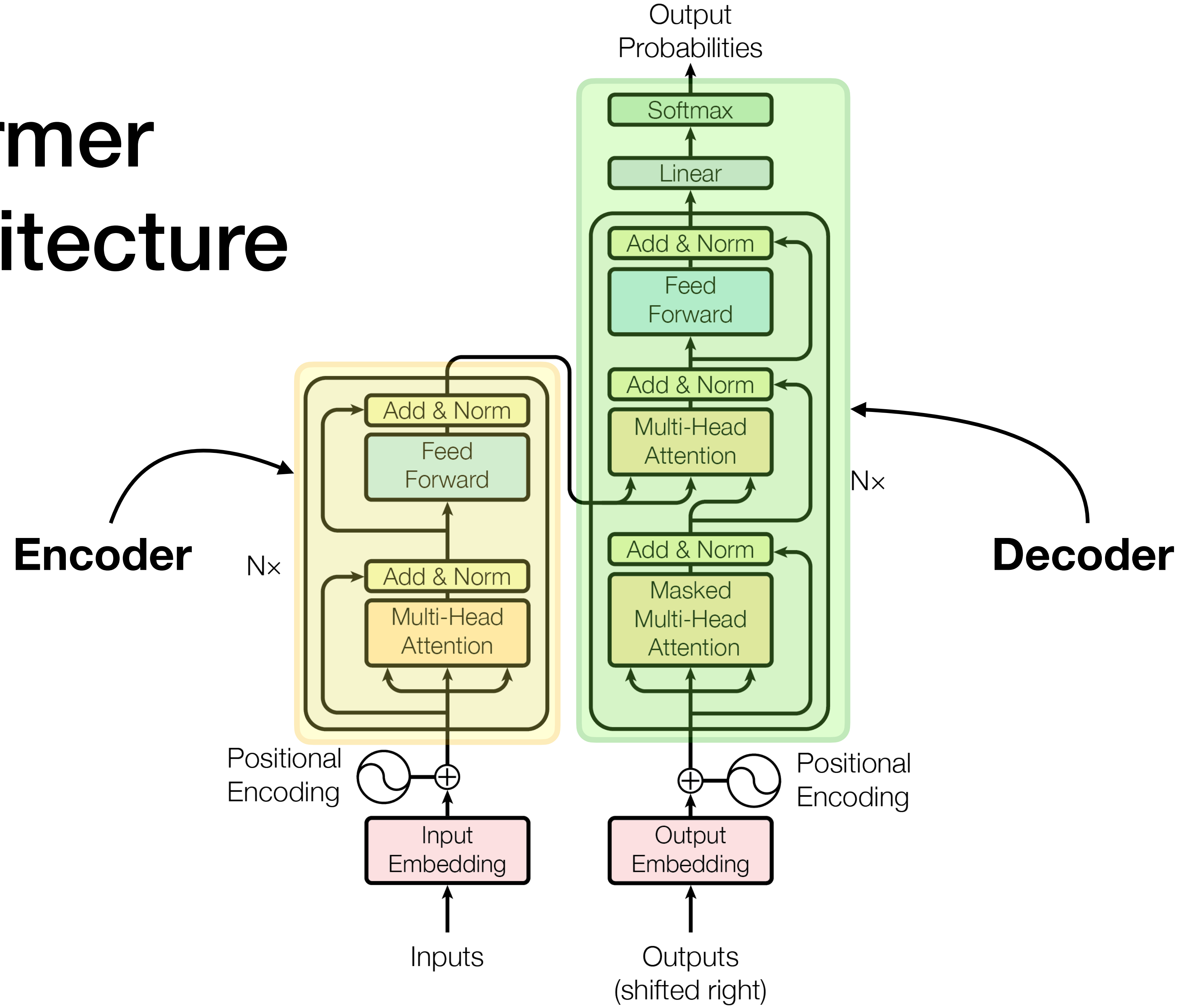
# Attention Is All You Need

- Scaled Dot-Product Attention
- Multi-head Attention
- Feed Forward Networks (FFNs)
- Positional Embeddings (PE)
- Self-attention v.s. cross-attention

Paper: [/https://arxiv.org/pdf/1706.03762](https://arxiv.org/pdf/1706.03762)

Visualization Github Repo: <https://github.com/jessevig/bertviz>

# Transformer Model Architecture

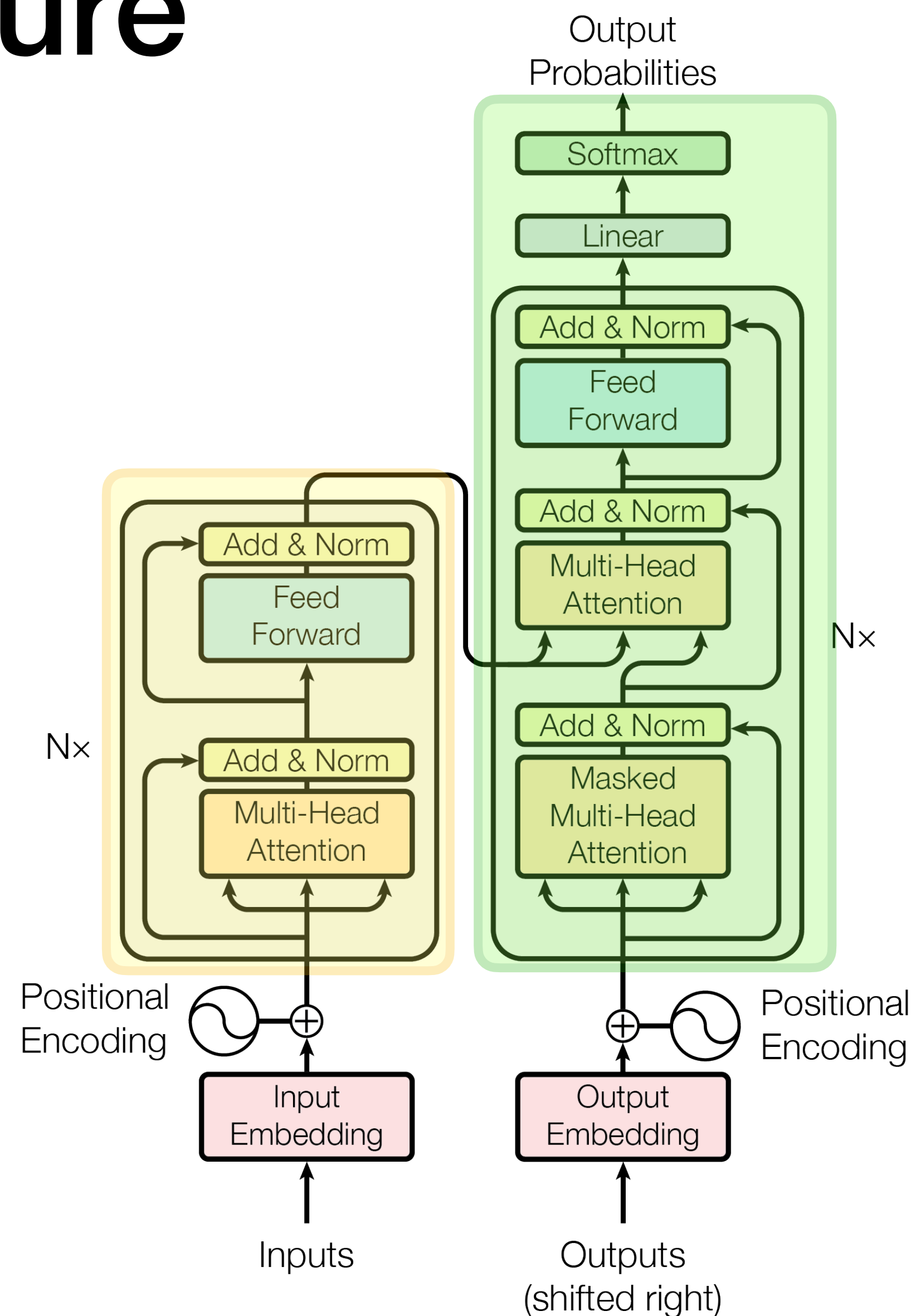


# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese

“I” “love” “this” “class” “!”  
Input

“\*” “\*” “\*” “\*” “\*”  
Output



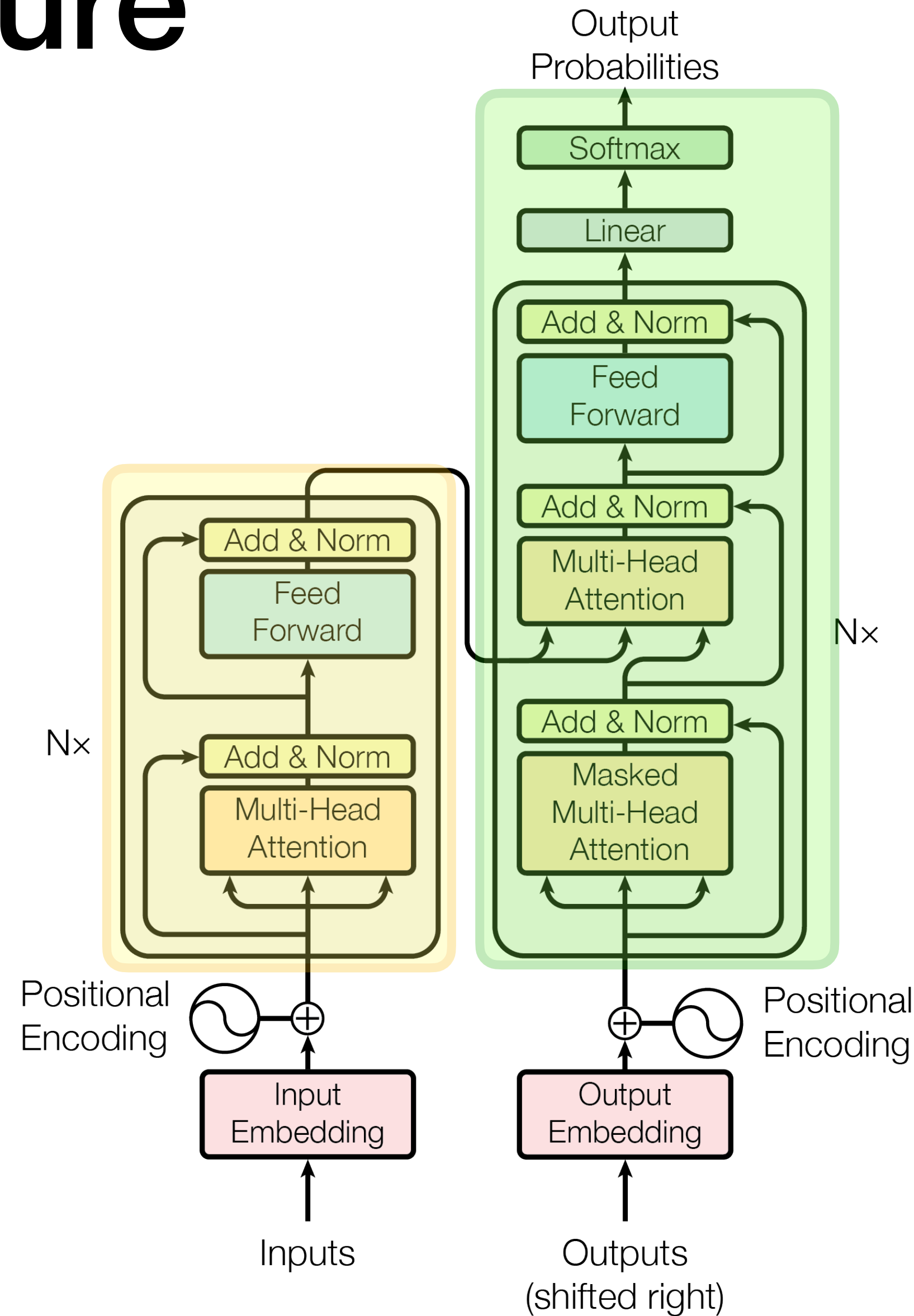
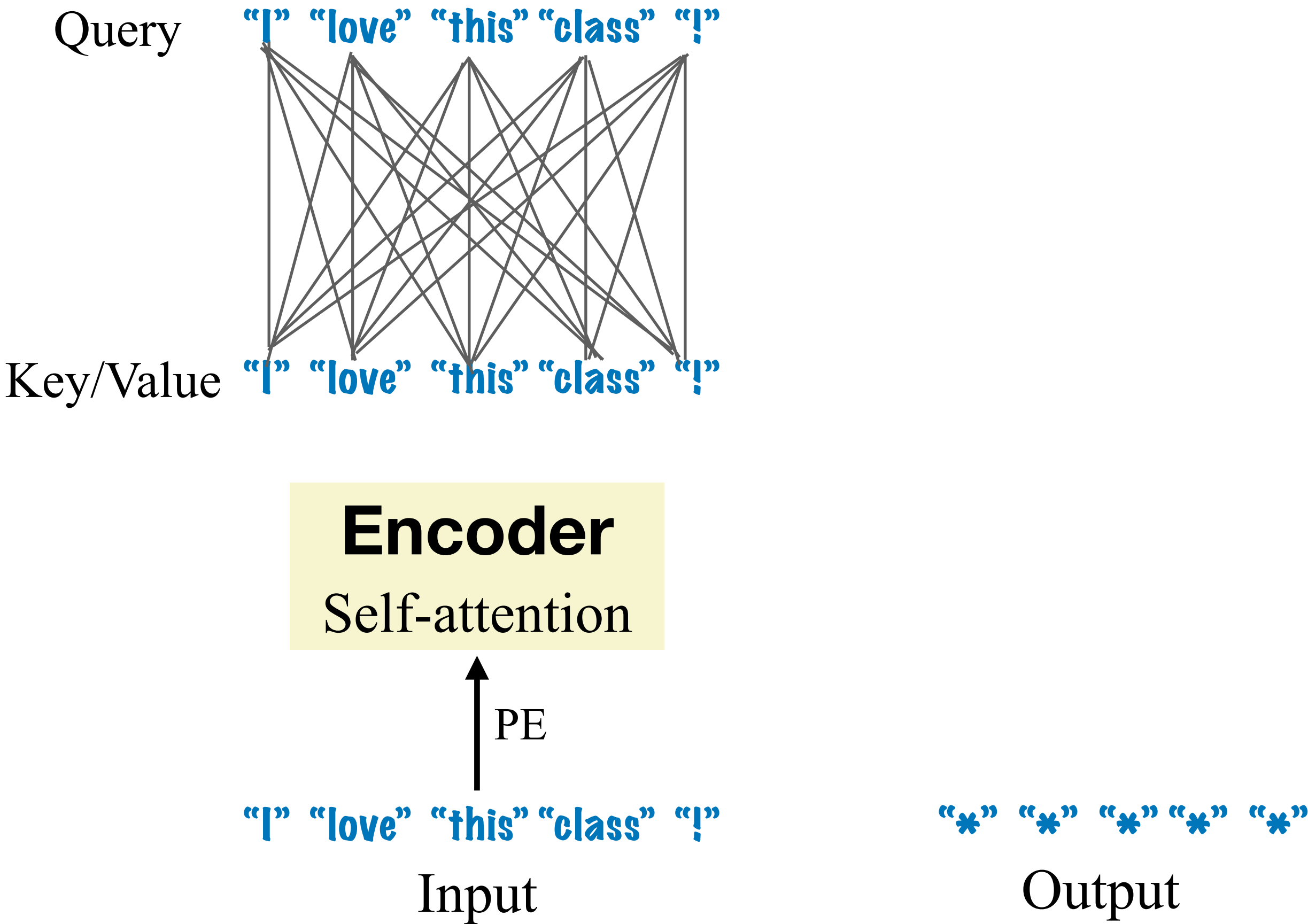
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



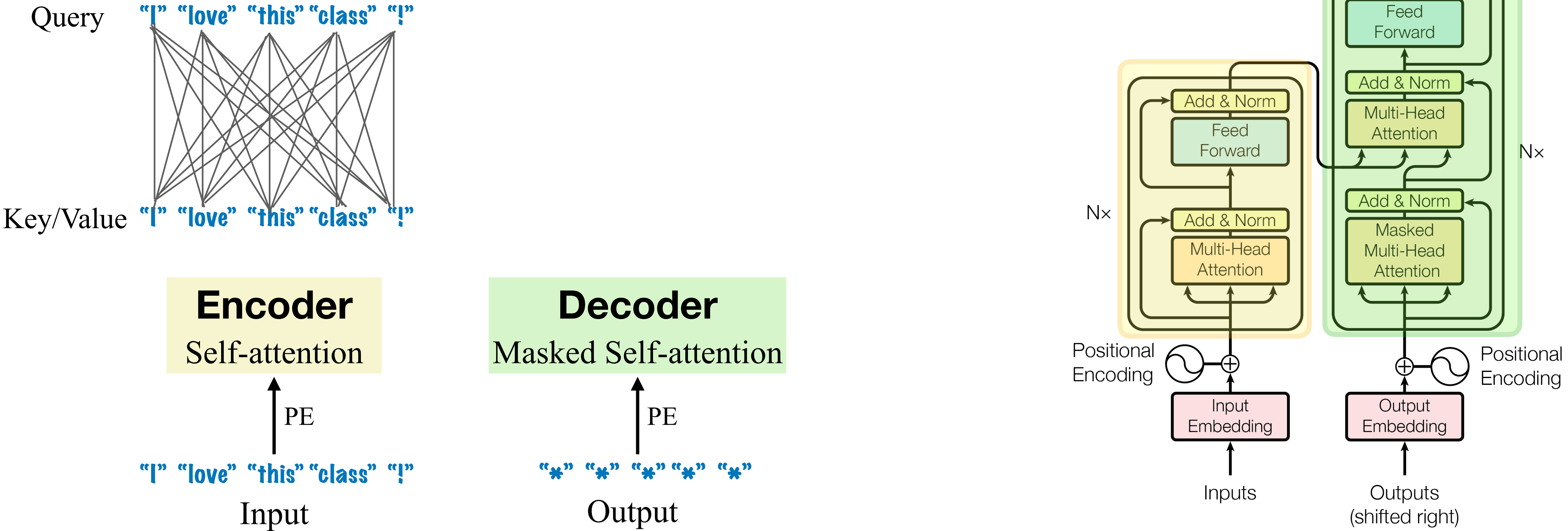
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



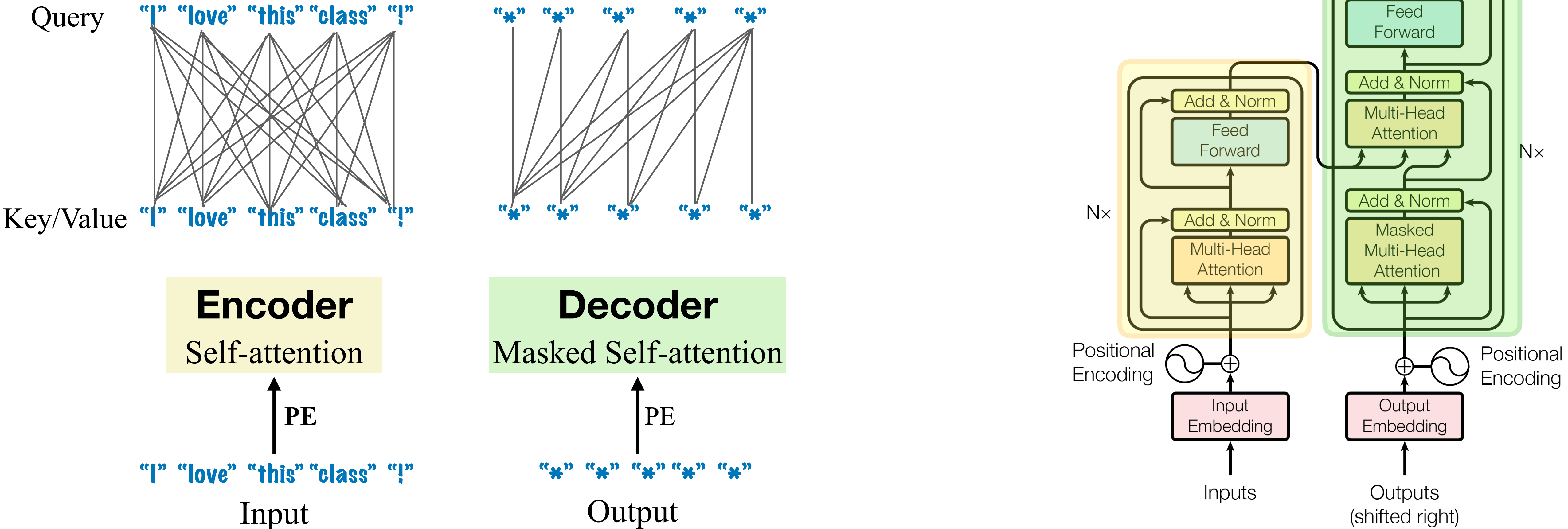
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



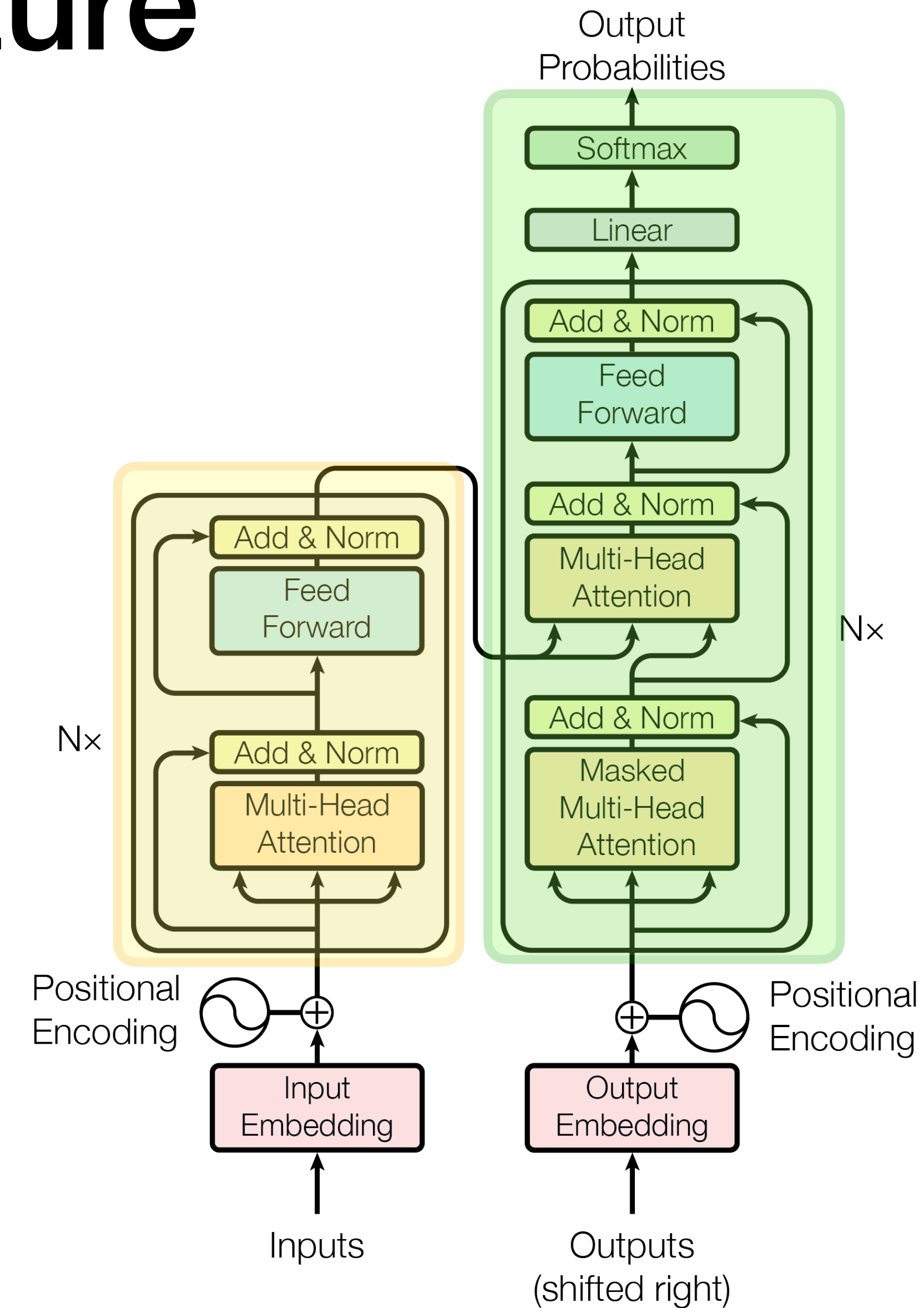
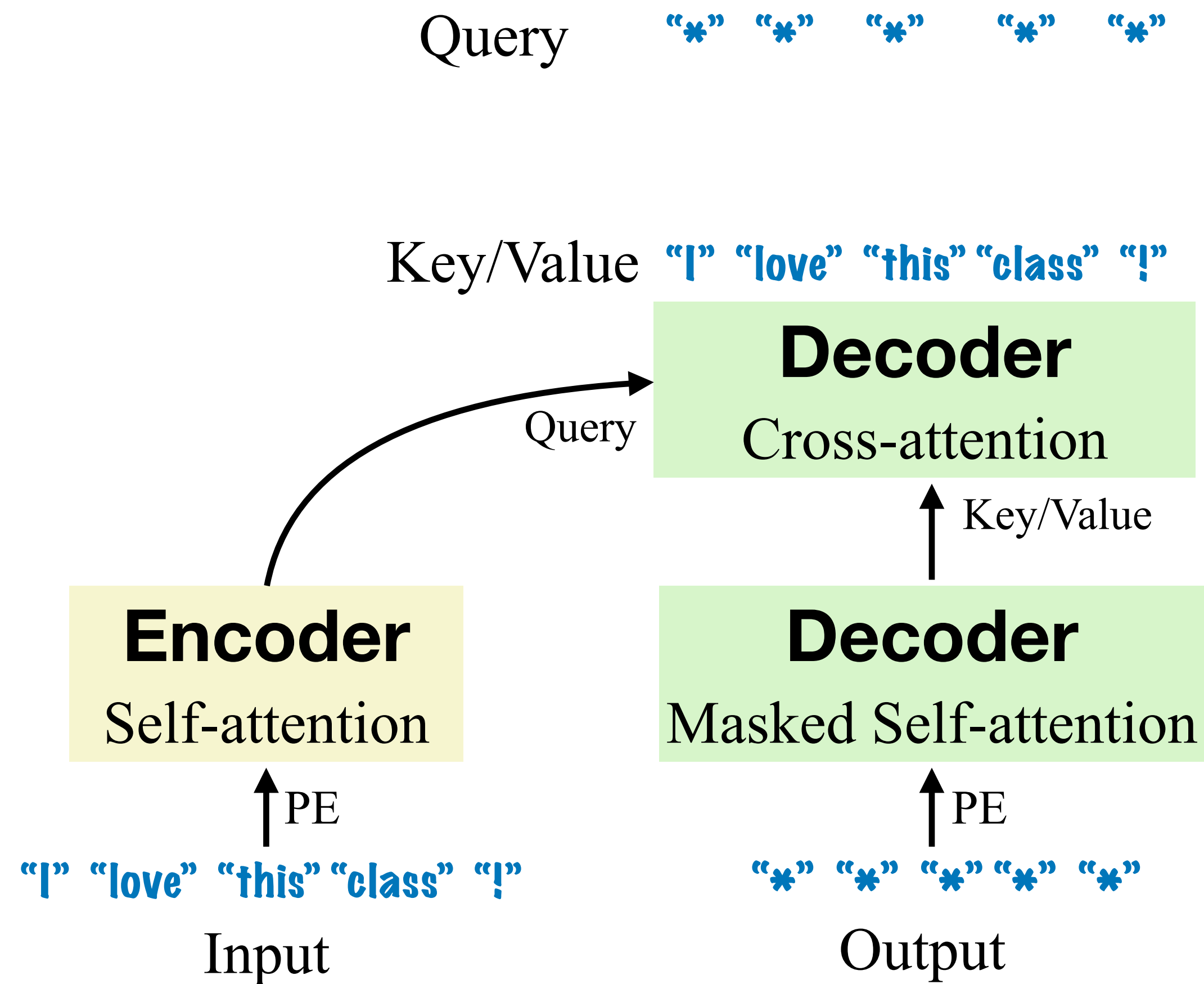
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



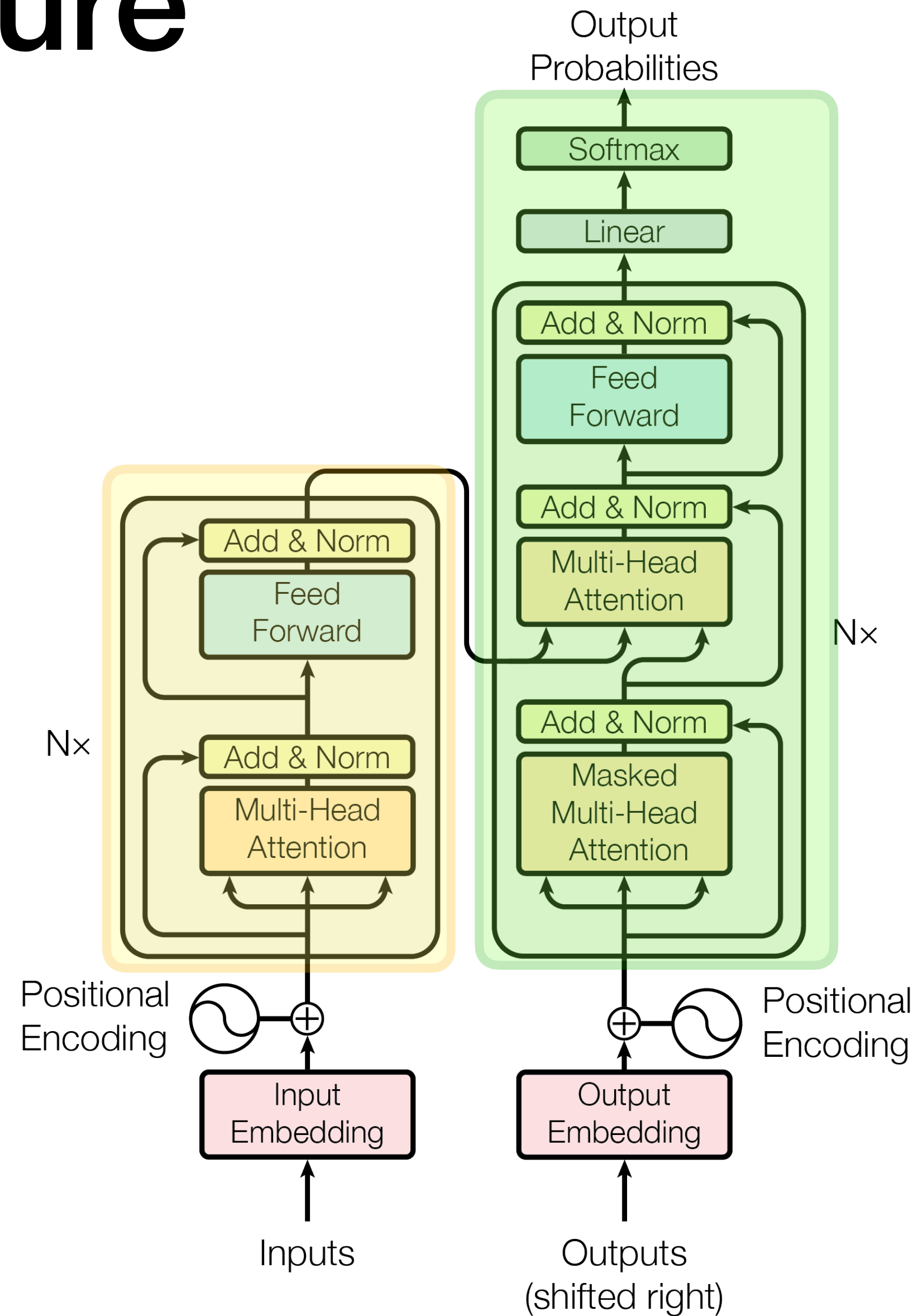
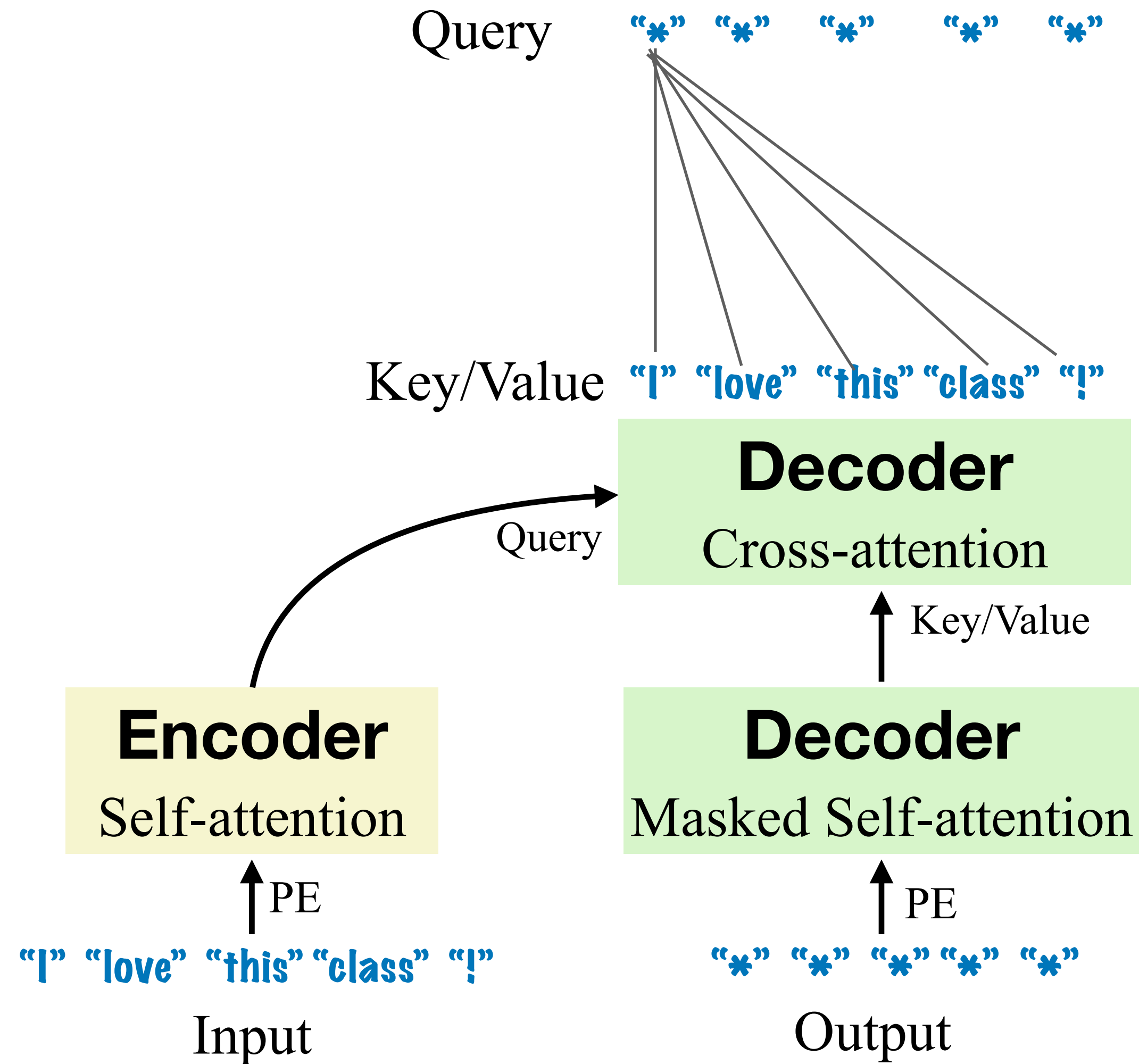
# Transformer Model Architecture

Translation task: English → Chinese



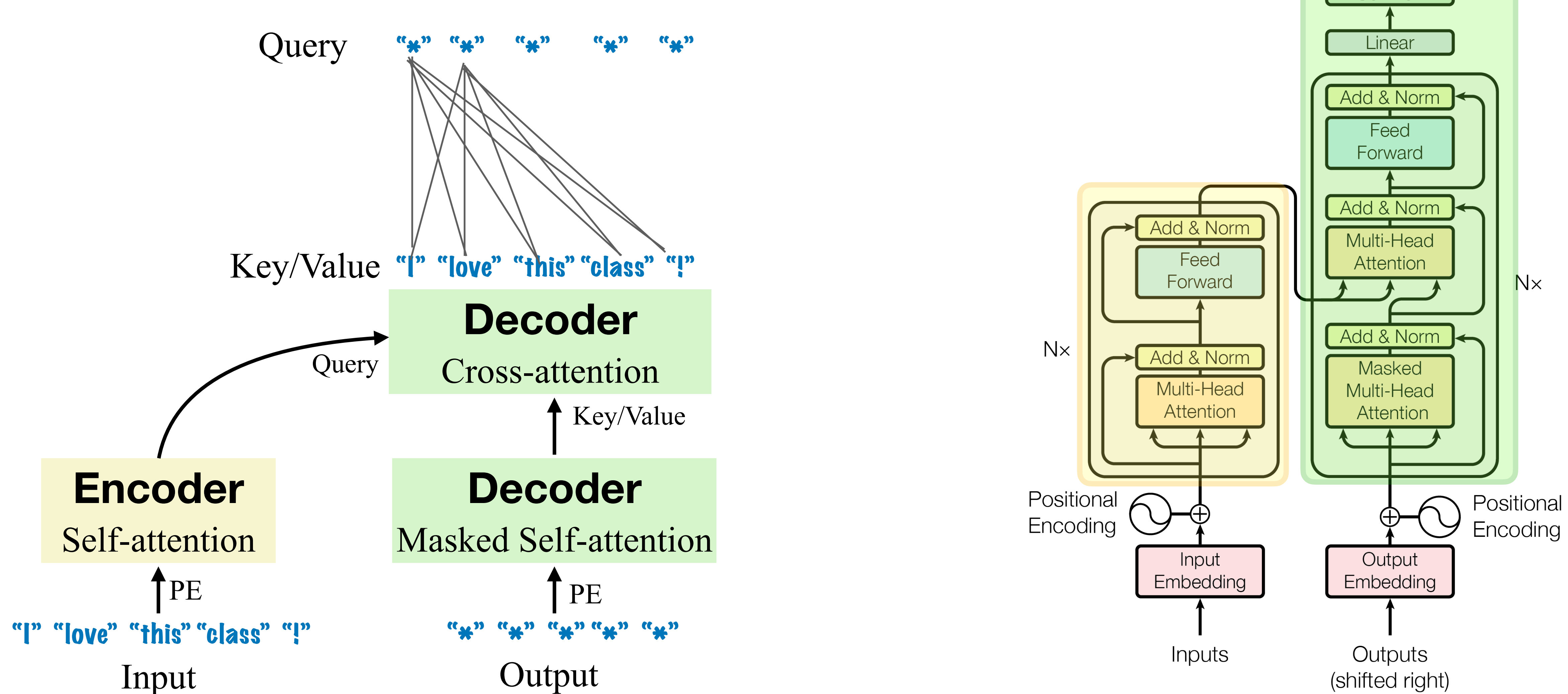
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



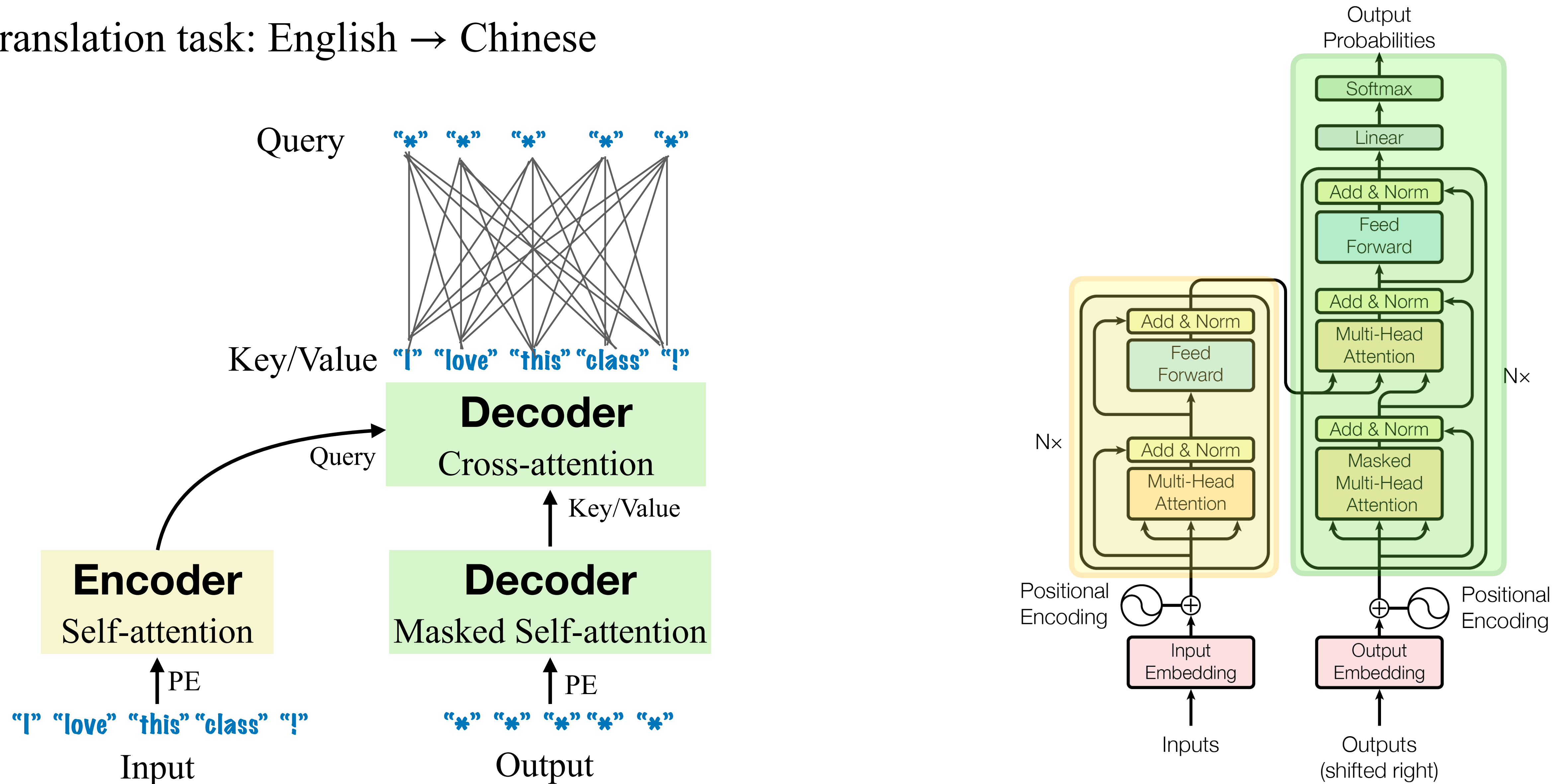
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese



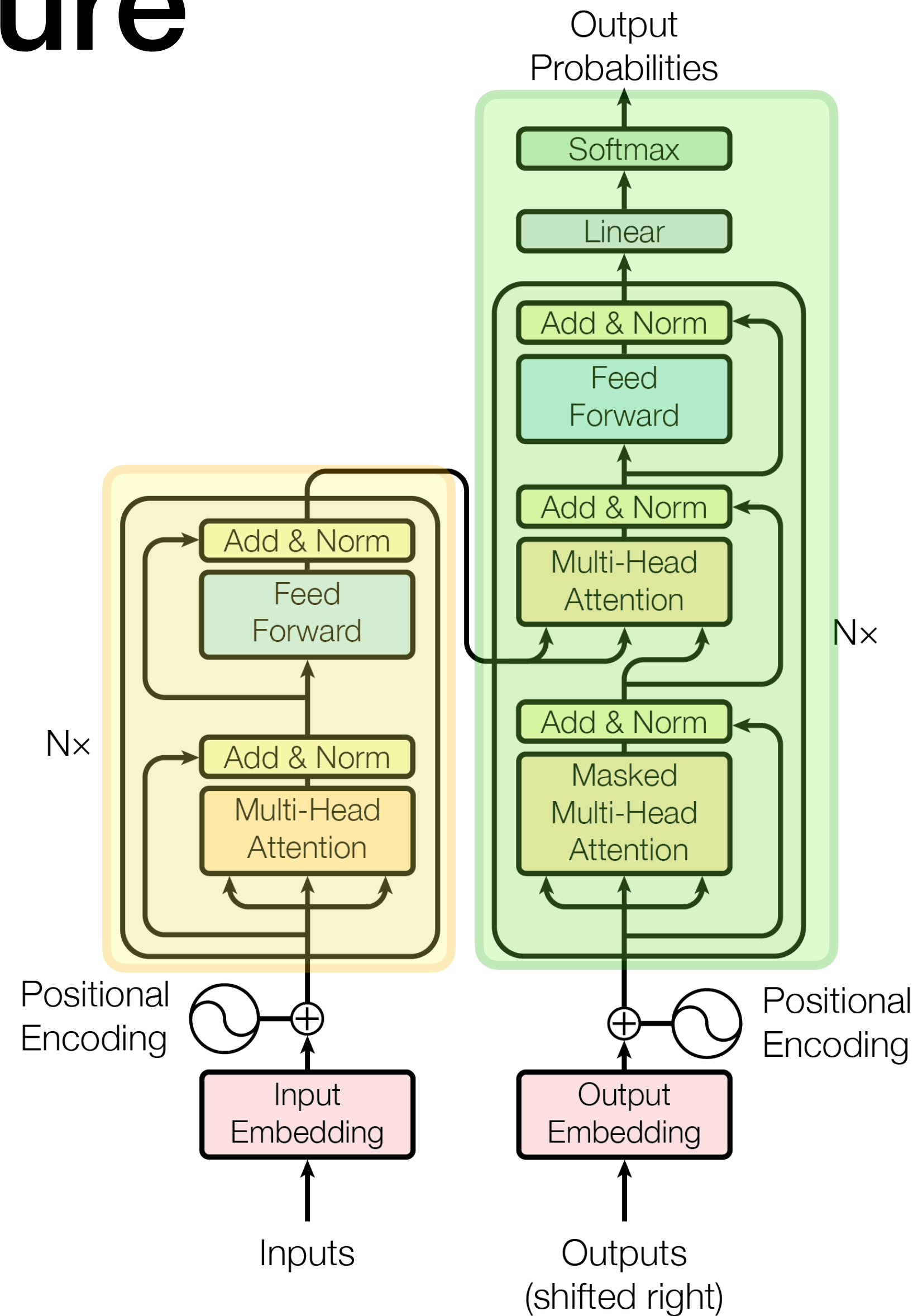
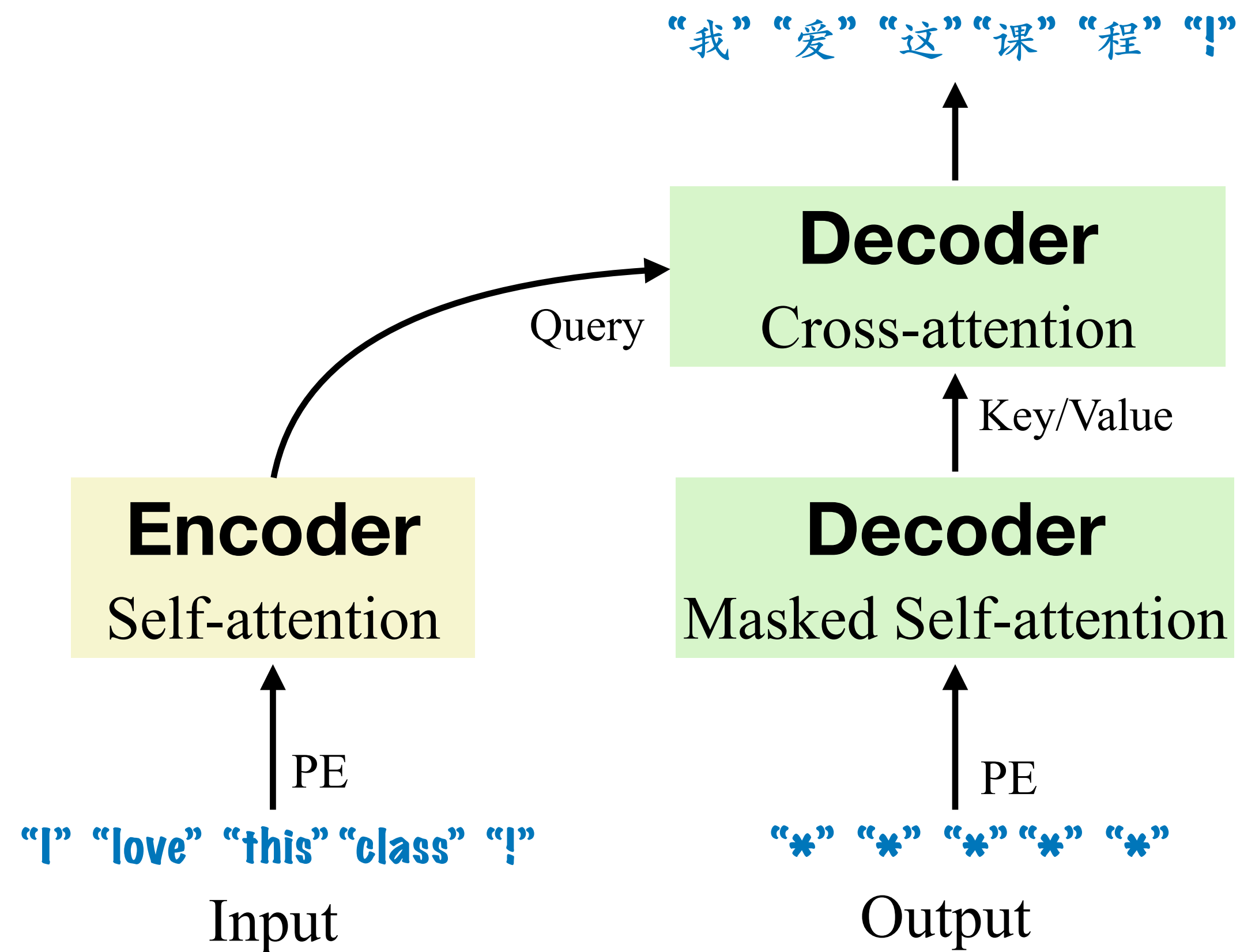
# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese

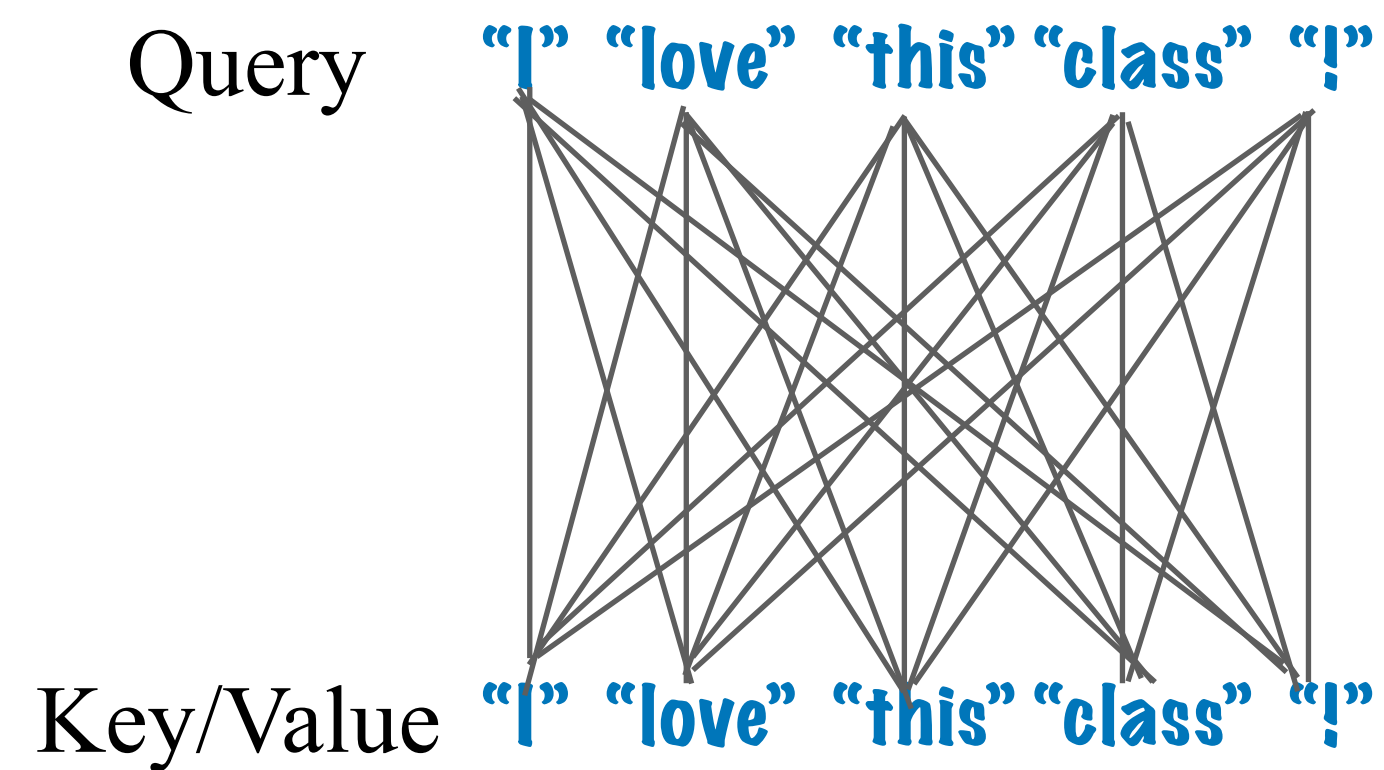


# Transformer Model Architecture

Translation task: English  $\rightarrow$  Chinese

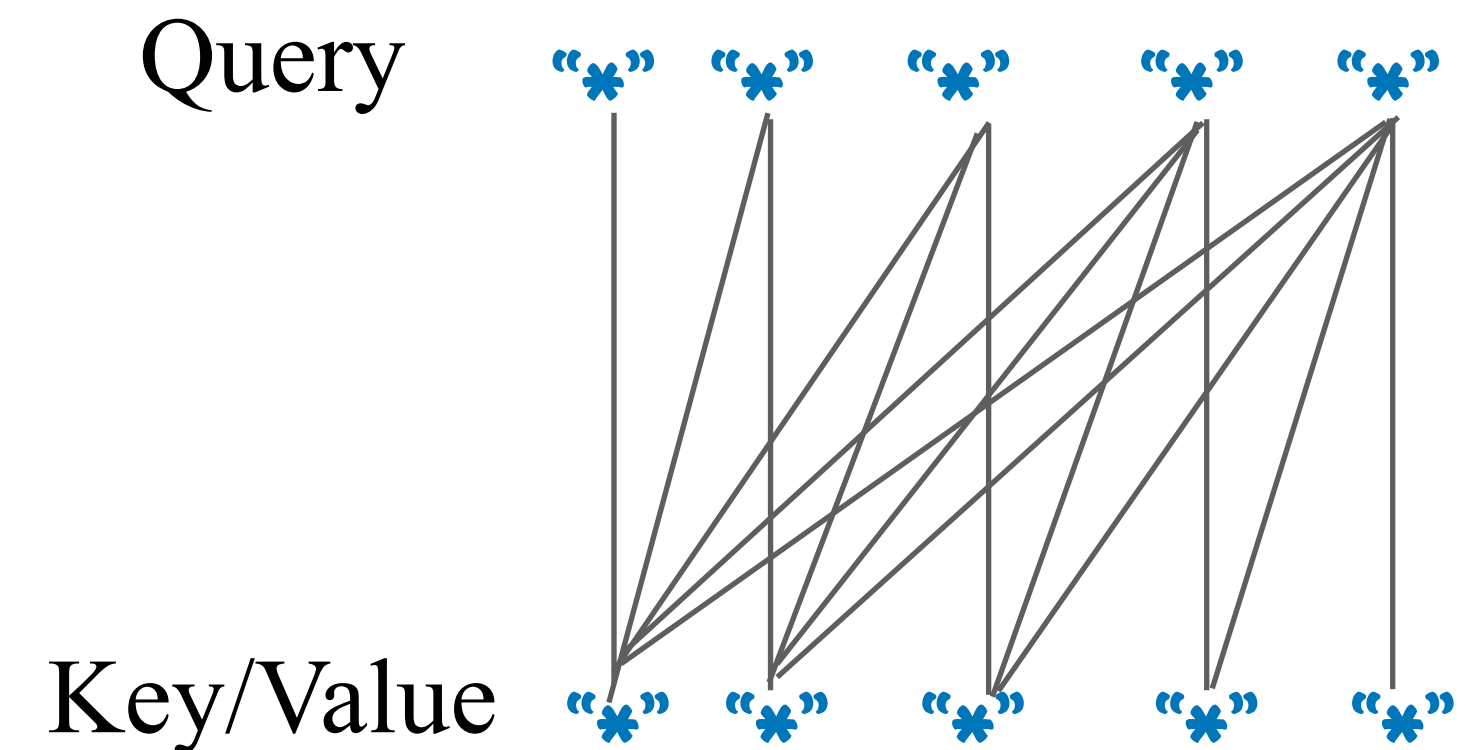


# Self-attention V.S. Masked self-attention



**Self-attention**

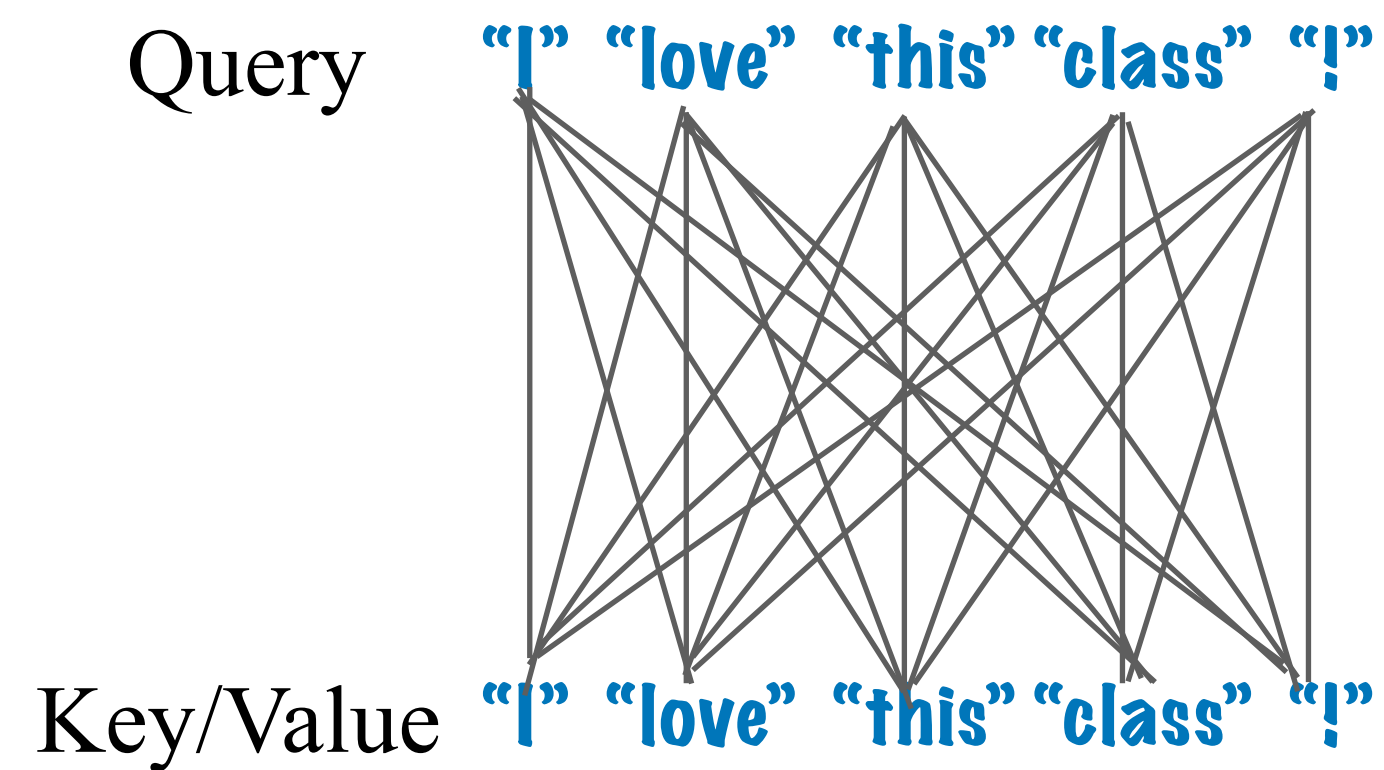
Query = Key/Value



**Masked self-attention**

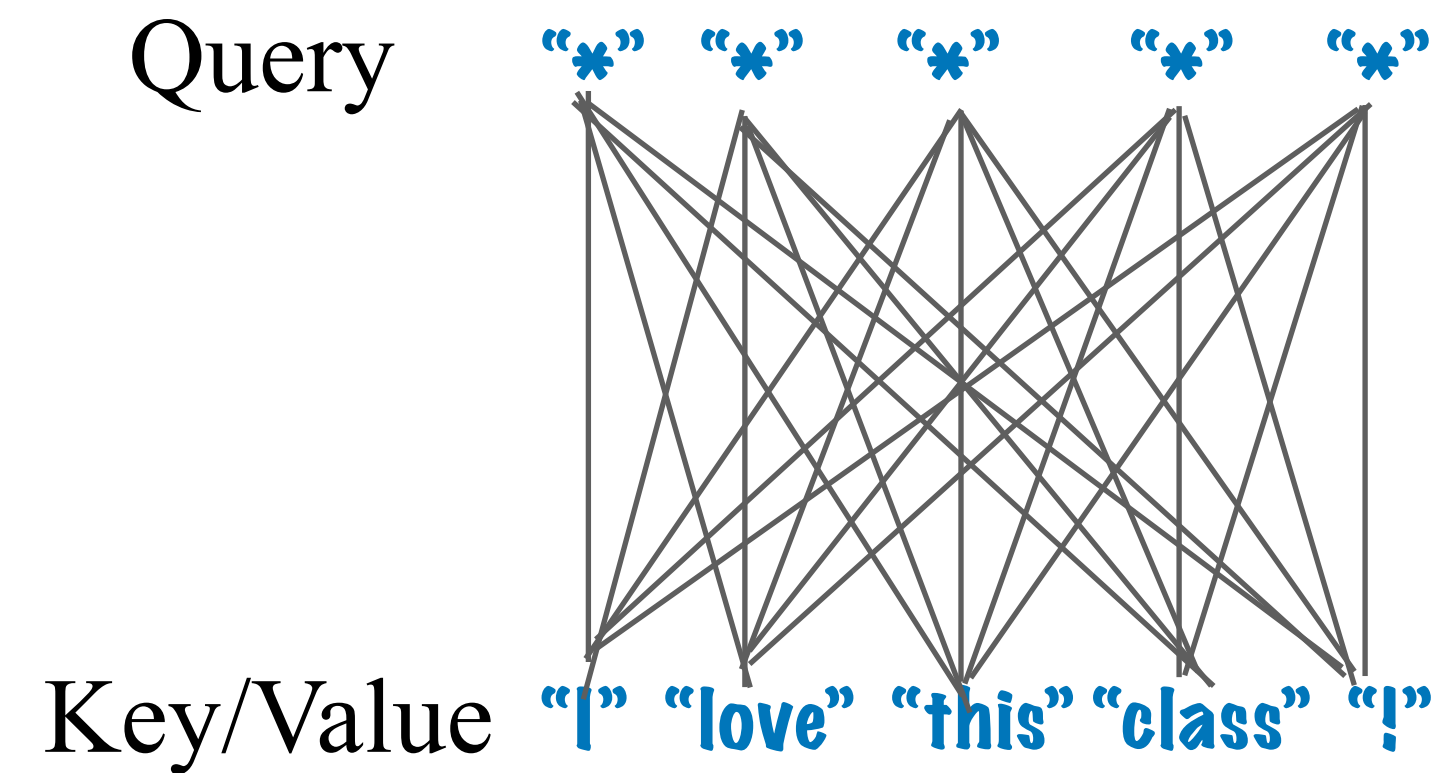
Query = Key/Value

# Self-attention V.S. Cross-attention



**Self-attention**

Query = Key/Value



**Cross-attention**

Query  $\neq$  Key/Value

# Transformer with attention

Visualization Github Repo: <https://github.com/jessevig/bertviz>

# Tokenization

Prompt:

Data visualization empowers users to ...

1

Token

Data

visualization

em

powers

users

to

# Tokenization

## Token Embedding

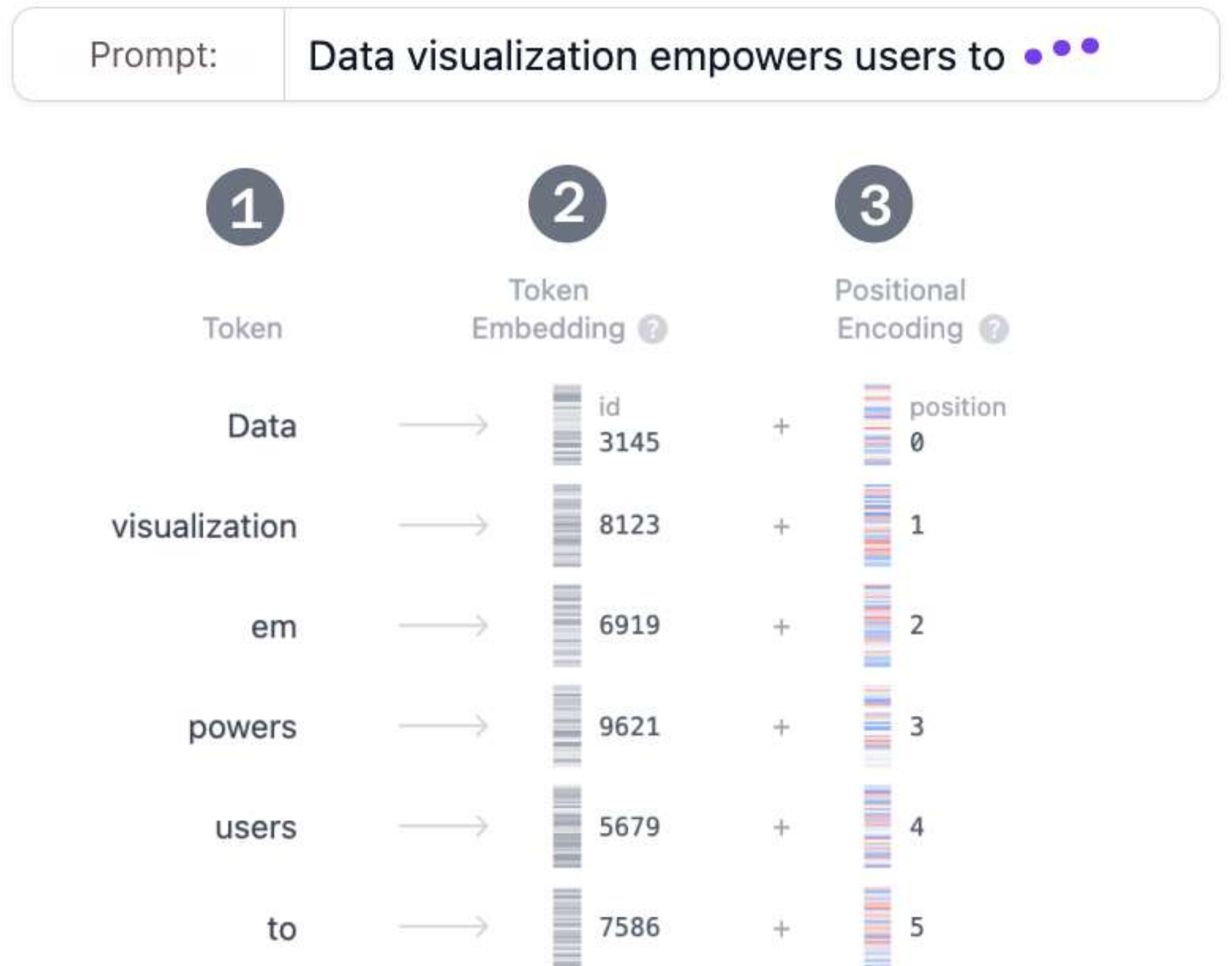
Prompt: Data visualization empowers users to ...



# Tokenization

## Token Embedding

## Positional Embedding



Tokenization

Token Embedding

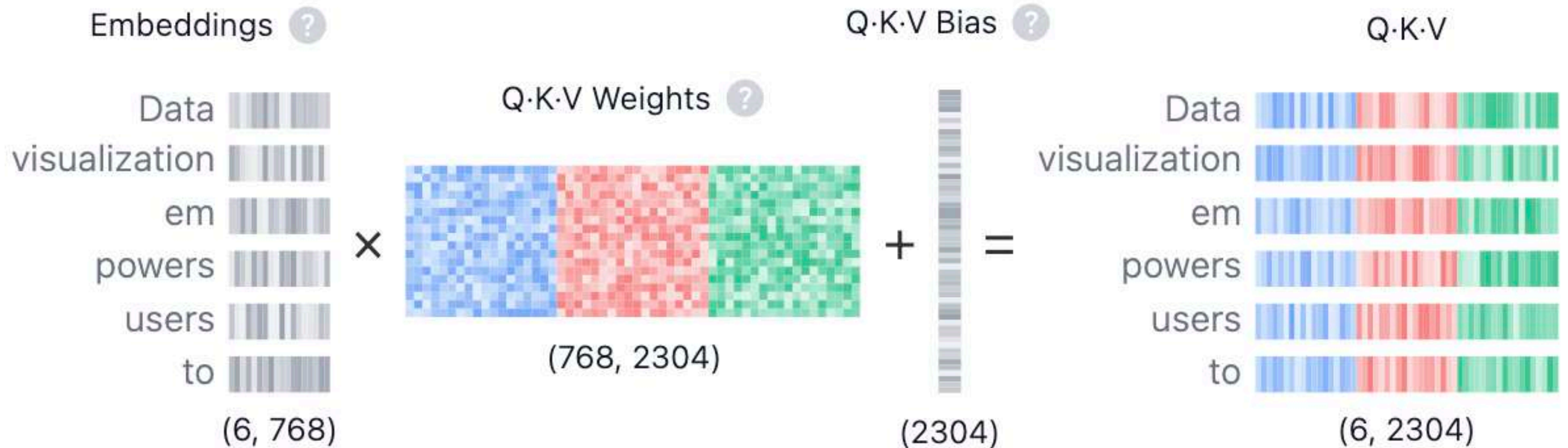
Positional Embedding

Input Embedding

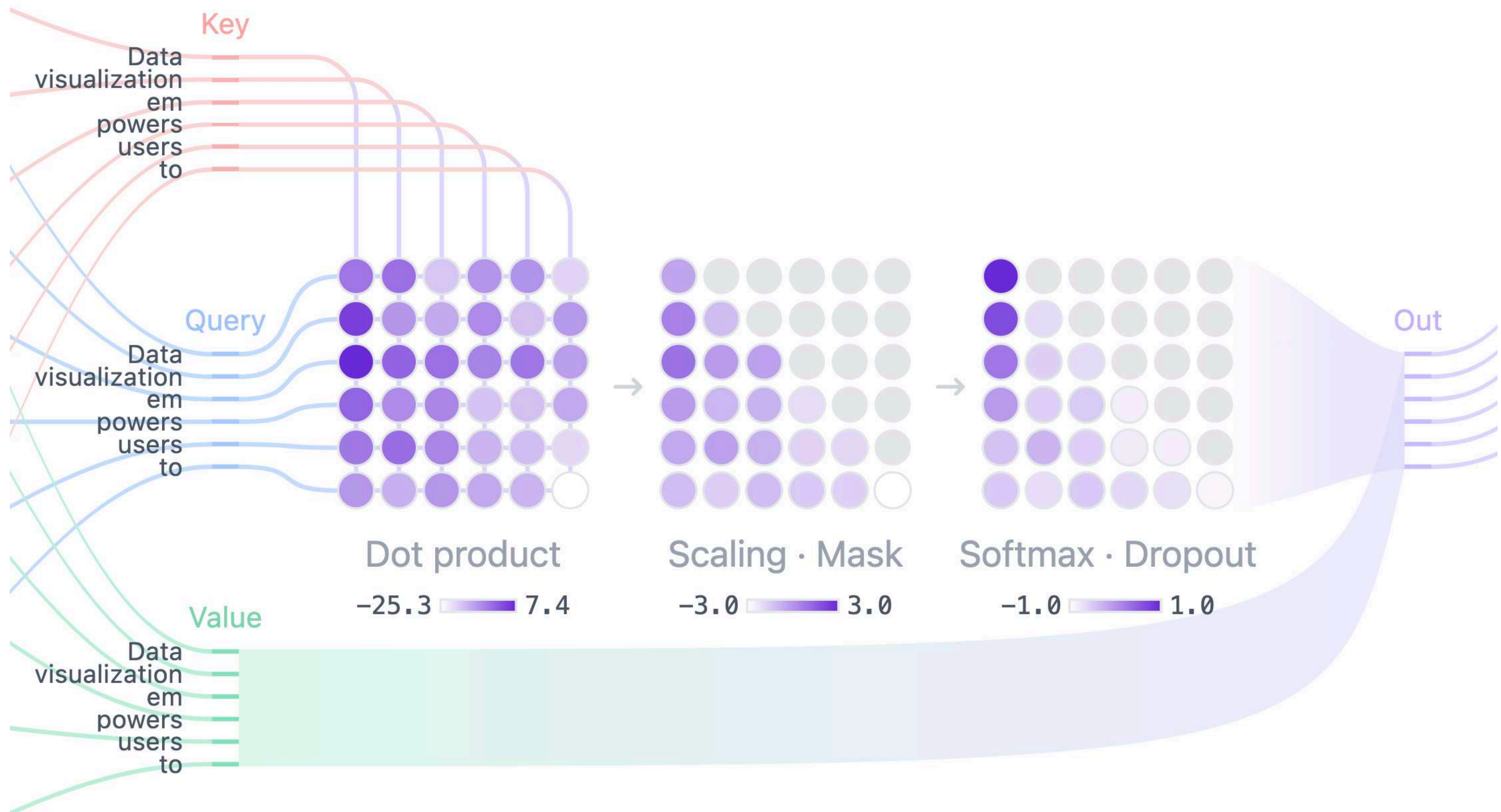
Prompt: Data visualization empowers users to ...



# Attention: Query, Key, Value



# Attention: Query, Key, Value



# Transformer Model Architecture

TRANSFORMER EXPLAINER

Examples ▾

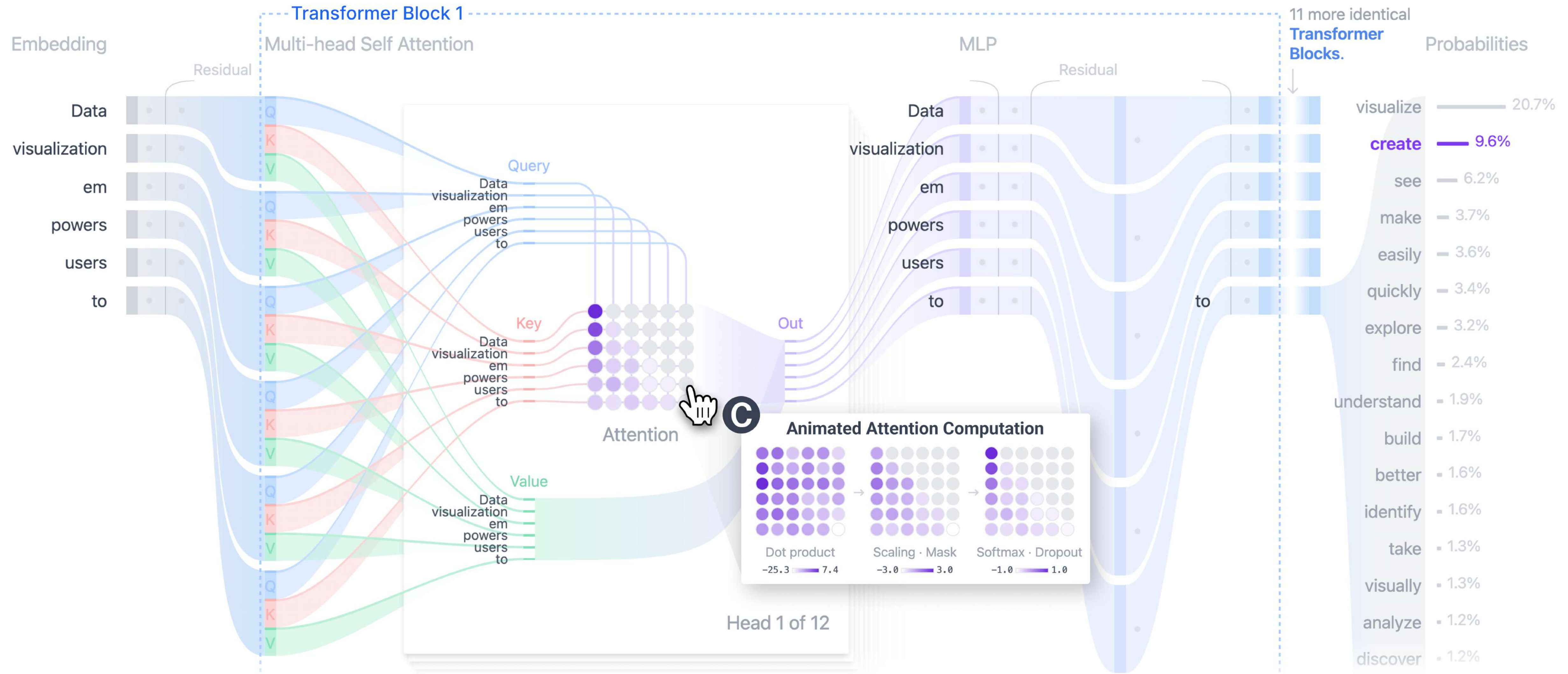
Data visualization empowers users to **create**

A

Generate

Temperature ?

B



# Vision Transformer

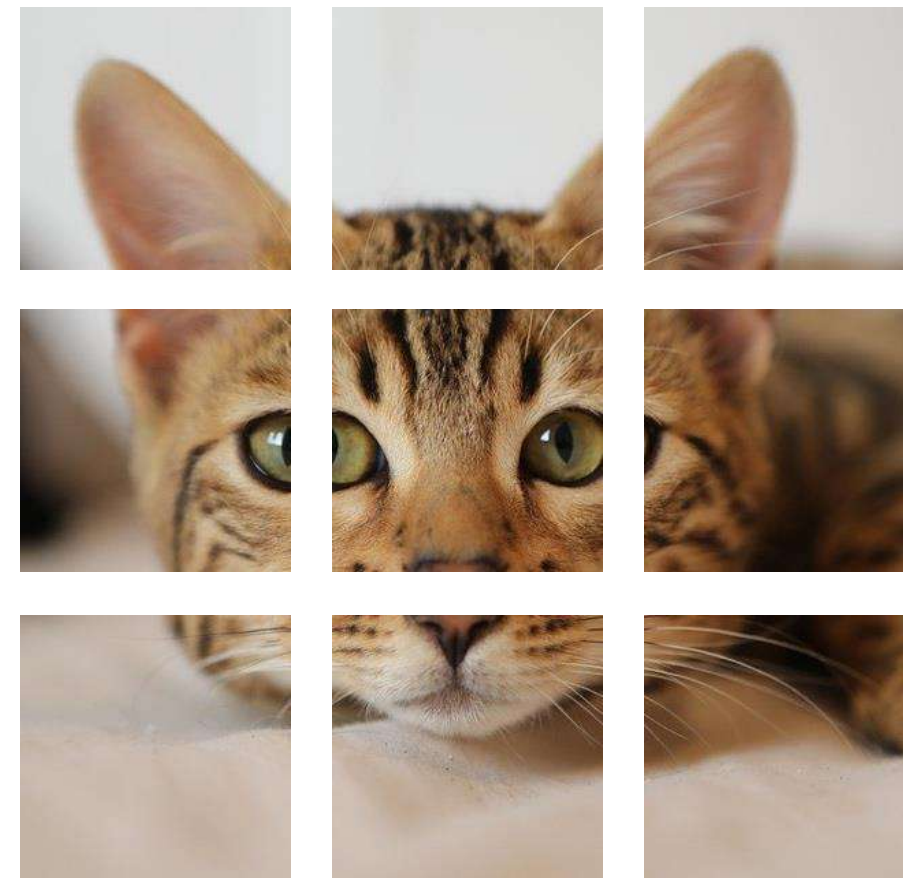
**An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**

Google Brain Team

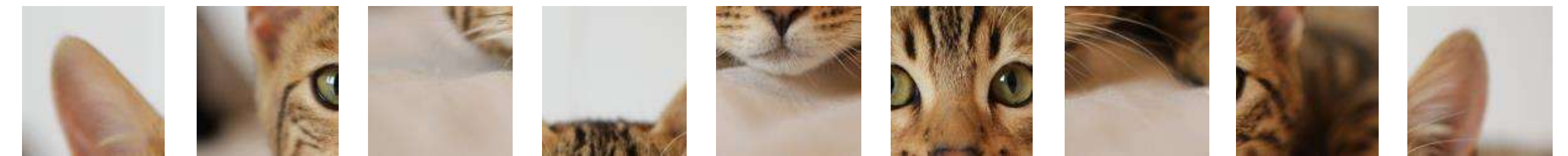
# Vision Transformers (ViT)



Input image:  
e.g. 224x224x3



N input patches with  
shape 3\*16\*16



2D image  $\rightarrow$  a sequence

# Vision Transformers (ViT)

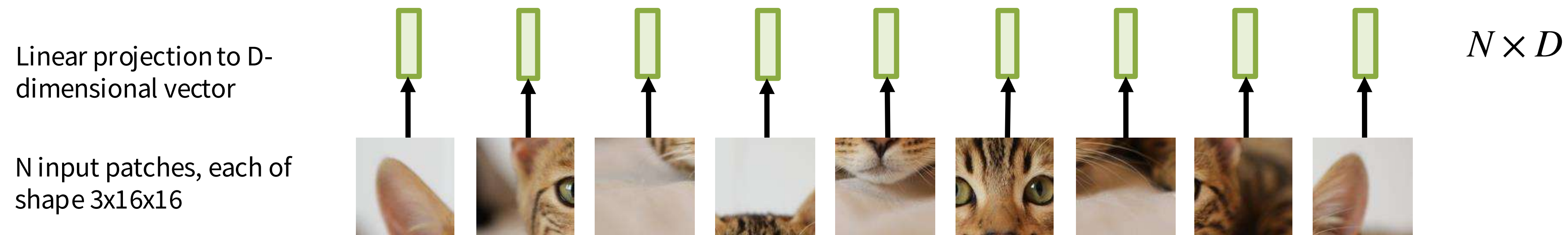
N input patches, each of  
shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

[Cat image](#) is free for commercial  
use under a [Pixabay license](#)

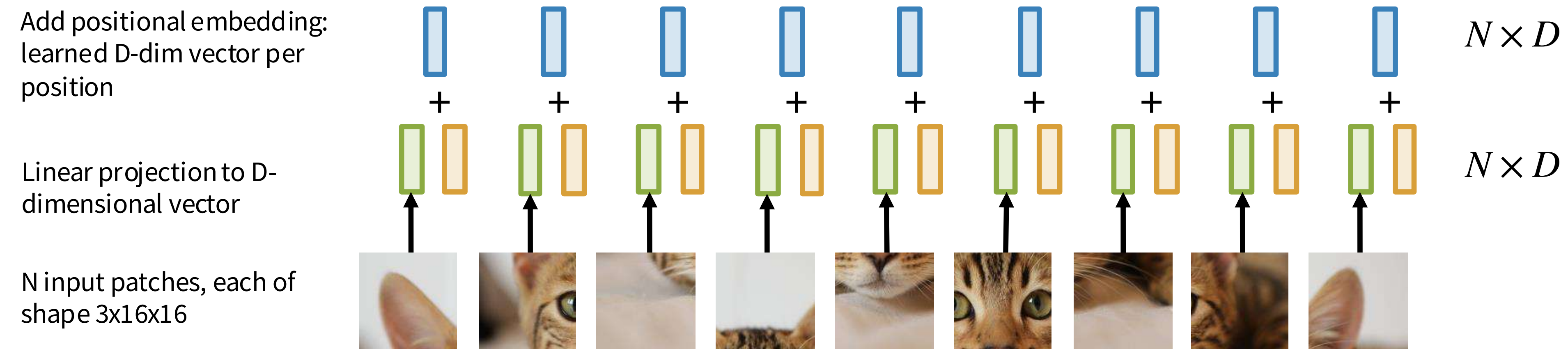
# Vision Transformers (ViT)



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

[Cat image](#) is free for commercial use under a [Pixabay license](#)

# Vision Transformers (ViT)

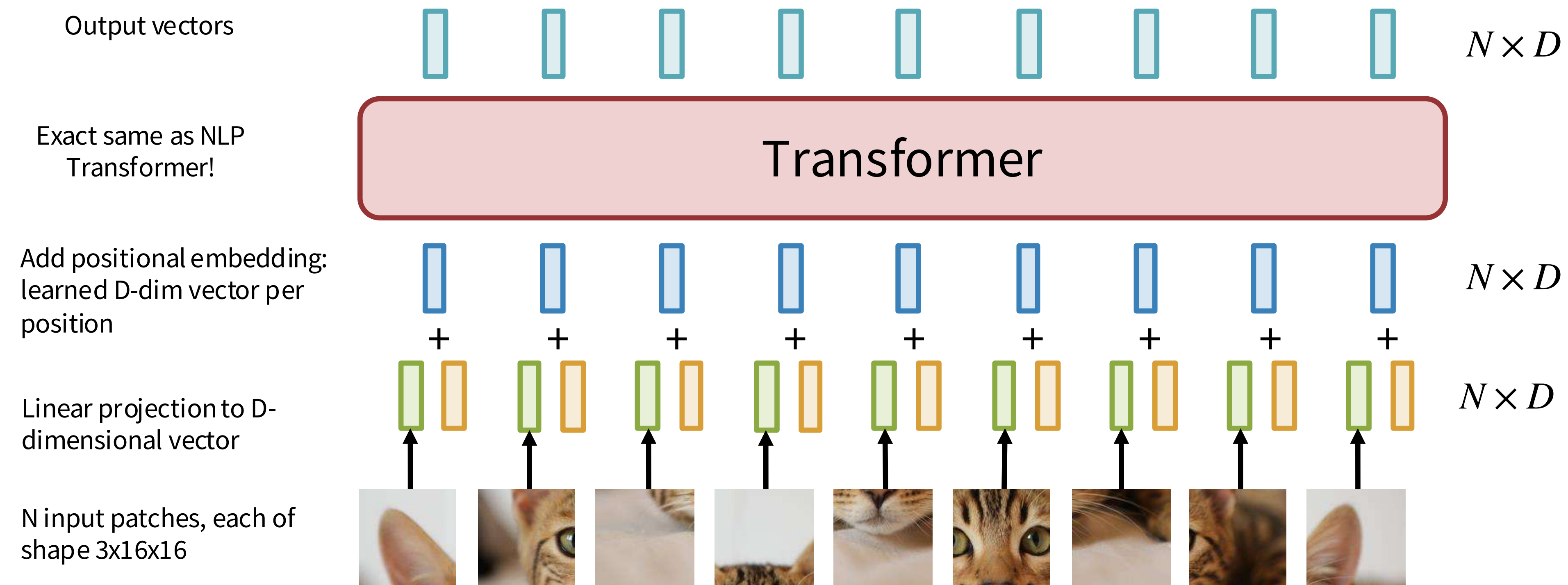


Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

[Cat image](#) is free for commercial  
use under a [Pixabay license](#)

# Vision Transformers (ViT)

**How to get the final c-dim classification score?**

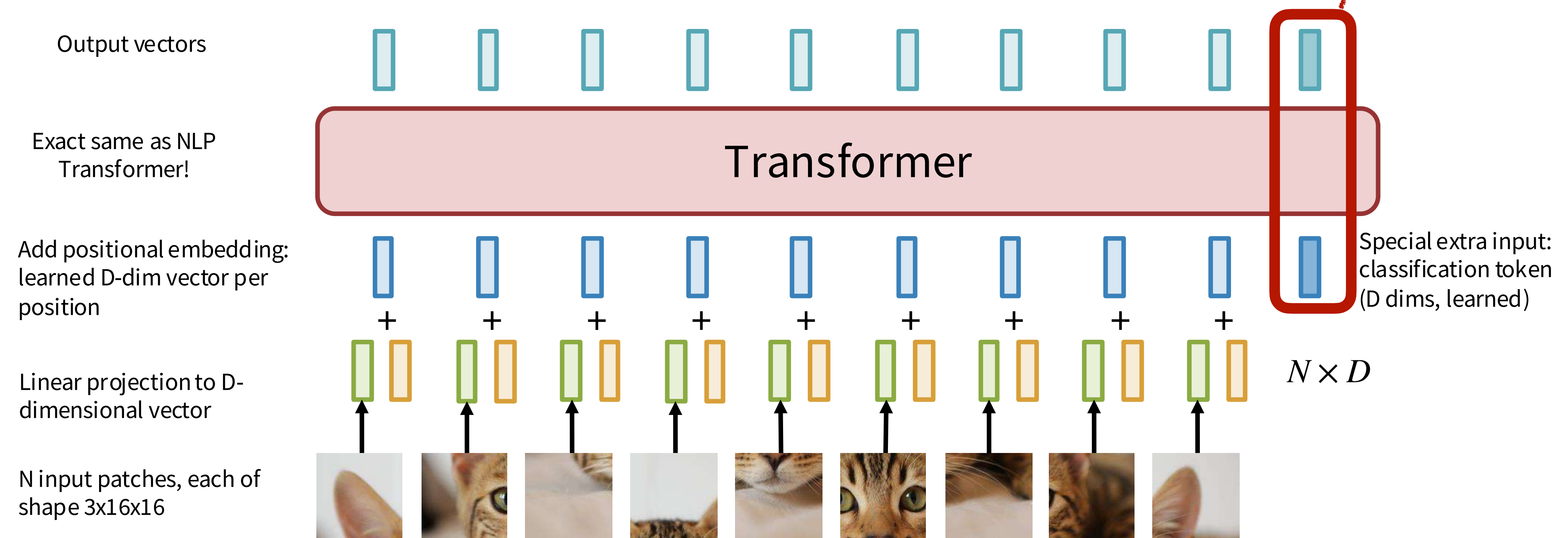


Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

[Cat image](#) is free for commercial use under a [Pixabay license](#)

# Vision Transformers (ViT)

**#1: a learnable  
classification token**

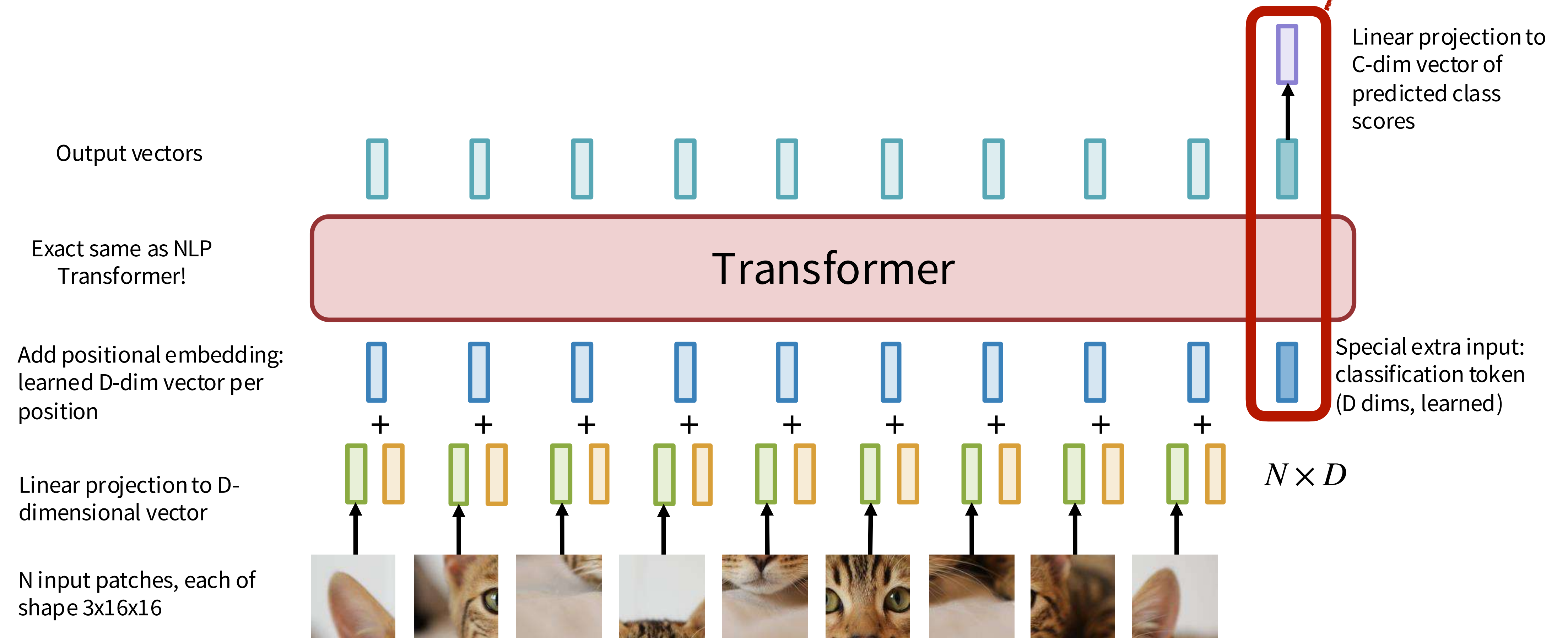


Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

[Cat image](#) is free for commercial use under a [Pixabay license](#)

**#1: a learnable  
classification token**

# Vision Transformers (ViT)

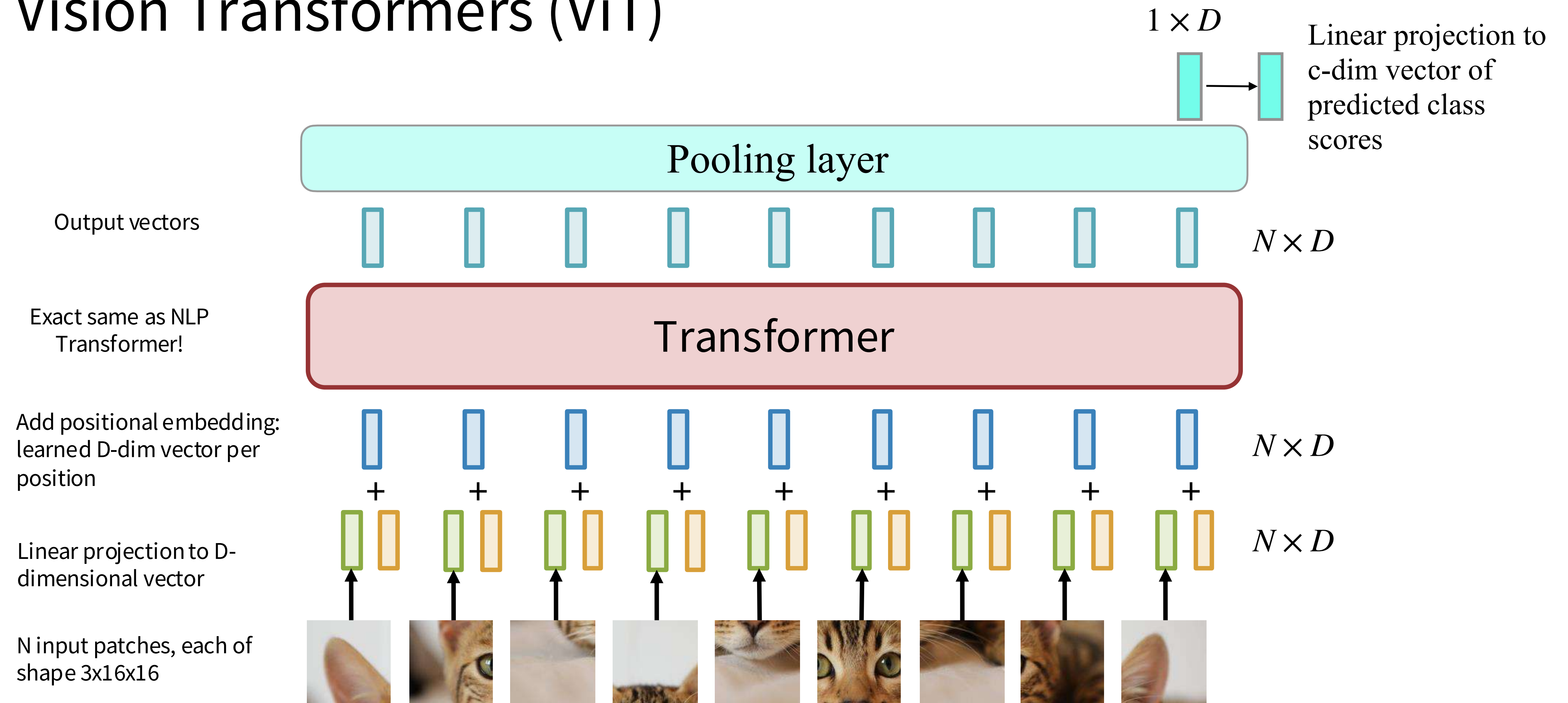


Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

[Cat image](#) is free for commercial use under a [Pixabay license](#)

# Vision Transformers (ViT)

## #2: a pooling layer



# Attention Visualization



2: Visualization of attention maps of the third Multi-Head Self-Attention block of ViT-Small 1 . We visualize the activations in attention maps of the **query patches (in the red box)**. The attention maps show that patches with high attention scores sparsely scatter around the query patch, and other patches have low attention scores.

[https://www.researchgate.net/publication/368290672\\_DilateFormer\\_Multi-Scale\\_Dilated\\_Transformer\\_for\\_Visual\\_Recognition](https://www.researchgate.net/publication/368290672_DilateFormer_Multi-Scale_Dilated_Transformer_for_Visual_Recognition)

# CSCI 598H Course Overview



**Weeks 2-4**

**Weeks 5-10**

**Weeks 11-16**

## **Module 1**

### **Deep Learning Basics**

- Linear Classifier and Optimization
- CNNs and BackPropagation
- RNNs, Attention, Transformers

## **Module 2**

### **Modern Visual Intelligence**

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

## **Module 3**

### **Engineering Systems**

- Robotics
- Earth science
- Medical imaging
- Energy

# CSCI 598H Course Overview



**Weeks 2-4**

**Weeks 5-10**

**Weeks 11-16**

## **Module 1**

### **Deep Learning Basics**

- Linear Classifier and Optimization
- CNNs and BackPropagation
- RNNs, Attention, Transformers

## **Module 2**

### **Modern Visual Intelligence**

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

## **Module 3**

### **Engineering Systems**

- Robotics
- Earth science
- Medical imaging
- Energy

# Next Lecture: Recognition & Localization

## Classification



Apple, **Burro**, Cat, Horse ...

## Object Detection



## Segmentation



**What / where is in the world?**